ATHO

Post-Quantum-Aware Proof-of-Work Digital Money

TECHNICAL WHITE PAPER

Protocol Architecture, Consensus Rules, and Reference Implementation

Revision 2 | 21 July 2026

Pre-production notice: Atho is experimental software under active development. This paper describes the current repository; it is not a mainnet launch approval, investment statement, or claim of completed independent security assurance.

Ghost Genull

genull@proton.me

Abstract

Atho is a post-quantum-aware, proof-of-work, public UTXO payment blockchain designed for durable settlement infrastructure. The project addresses a practical systems problem: a public value-transfer network must remain independently verifiable, operationally simple, and cryptographically conservative while also planning for long-horizon changes in signature-security assumptions. Atho uses a public UTXO accounting model, local full-node validation, SHA3-384 proof-of-work hashing, Falcon-512 transaction signatures, Rust-first systems implementation, and network-specific consensus isolation.

The present codebase is organized around protocol types, consensus logic, block and UTXO persistence, Falcon cryptography, wallet state, peer networking, API surfaces, mining logic, and desktop-client interaction. The current implementation uses 100-second blocks, 8 decimal places, 1-confirmation normal transaction consensus spendability, wallet-selected confirmation policy, 100-block coinbase maturity, a zero-emission genesis block at height `0`, ordinary subsidy activation at height `1`, a four-era bootstrap schedule of `8`, `4`, `2`, and `1` ATHO for `1,250,000` blocks each, a permanent `0.50 ATHO` tail reward from height `5,000,001`, no premine, a required transaction fee floor of `max(600 atoms, 1 atom per serialized byte)`, optional fee above that floor as miner tip, mandatory TX-PoW that scales from `10` to `16` bits, and an adaptive block-size rule that launches at 1,000,000 bytes and stays bounded by the 10,000,000-vbyte / 10,000,000-raw-byte hard ceilings. The repository includes stricter transaction decoding, witness commitment behavior, explicit mainnet/testnet mining reward address requirements, bounded mempool configuration, storage failpoints, replay-oriented UTXO integrity checks, and property-test coverage. This paper explains Atho's architecture, transaction model, ownership rules, block structure, mining flow, monetary behavior, wallet architecture, Base56 address encoding, storage layout, synchronization model, threat model, performance model, testing strategy, and upgrade philosophy. It distinguishes live implementation facts from earlier planning claims and treats the repository as the source of truth.

Cryptographic Context and Scope

Many established cryptocurrency systems rely on elliptic-curve signatures such as ECDSA or Schnorr variants. Those signature families remain practical and widely deployed today, but long-term cryptographic planning benefits from acknowledging that sufficiently capable quantum systems running algorithms such as Shor's algorithm would pose a structural threat to elliptic-curve signature assumptions. That is not a claim that Bitcoin or other elliptic-curve systems are presently broken in the real world. It is a statement about long-horizon protocol planning.

Atho addresses transaction authorization with Falcon-512, a lattice-based post-quantum signature scheme. The network also uses SHA3-384 for proof-of-work and hashing commitments. Together, those choices preserve a classical proof-of-work operating model while reducing dependence on elliptic-curve signatures for ownership authorization. The design goal is not to make exaggerated claims about permanent immunity. The design goal is to build a payment chain whose authorization model is more resilient to known quantum-era signature concerns than a conventional ECC-based ledger.

Keywords: Atho, Falcon-512, SHA3-384, proof-of-work, public UTXO, Rust, post-quantum cryptography, payment network, digital money

Table of Contents

Executive Summary

This white paper summarizes the repository's current implementation of Atho as a post-quantum-aware proof-of-work payment network. The codebase combines deterministic full-node validation, a public UTXO accounting model, SHA3-384 proof-of-work, Falcon-512 transaction authorization, and strict network separation across mainnet, testnet, regnet, and prunetest.

The current implementation most directly exposes the following operator-relevant facts:

- target block time: 100 seconds
- normal transaction consensus spendability: 1 confirmation
- default wallet confirmation filter: 3 confirmations
- coinbase maturity: 100 blocks
- zero-emission genesis anchor at height `0`
- first ordinarily mineable subsidy height: `1`
- emission model: four bootstrap eras paying `8`, `4`, `2`, and `1` ATHO for `1,250,000` blocks each, followed by a permanent `0.50 ATHO` tail reward
- bootstrap issuance at tail activation: `18,750,000 ATHO`
- no premine
- no maximum supply limit
- first-era annualized base issuance: `2,522,880 ATHO`
- permanent tail annualized base issuance: `157,680 ATHO`
- accounting precision: 8 decimals, where 1 ATHO = 100,000,000 atoms
- required transaction fee: `max(600 atoms, 1 atom per serialized byte)`
- optional miner tip guidance: users may add fee above that floor and wallets may suggest higher priority rates when congestion warrants it
- dust threshold: 100 atoms, equal to 0.00000100 ATHO
- TX-PoW anti-spam: deterministic 10-to-16-bit sender work based on transaction shape
- block budget: adaptive 1,000,000-byte launch limit, bounded by 10,000,000 vbytes, 40,000,000 weight units, and 10,000,000 raw serialized bytes
- signature and hashing primitives: Falcon-512 for transaction authorization and SHA3-384 for proof-of-work
- network isolation: distinct consensus IDs, address prefixes, P2P magic values, genesis blocks, and bootstrap configuration
- mining policy: explicit same-network reward address requirement on mainnet and testnet
- mempool policy defaults: 50,000 transactions and 64 MiB of virtual transaction data
- node access controls: loopback RPC defaults with cookie or HMAC authentication and explicit public-RPC gating

The rest of the paper explains how those rules appear in consensus, storage, networking, mining, wallet behavior, and operator-facing interfaces. Atho keeps all consensus accounting in integer atoms, uses a

zero-emission genesis anchor, pays `8`, `4`, `2`, and `1` ATHO across four bootstrap eras of `1,250,000` blocks each, then clamps permanently to a `0.50 ATHO` tail reward instead of repeatedly right-shifting toward zero. Miners may claim the active subsidy plus included transaction fees, where every non-coinbase transaction must satisfy the consensus fee floor and may optionally add extra tip value above it.

1. Introduction

Digital money is most useful when it is understandable, independently verifiable, and difficult to corrupt. Bitcoin established the proof-of-work timestamp-chain and UTXO model that informs Atho's payment-oriented architecture [2]. Over time, many blockchain systems expanded into large execution environments with broad virtual-machine surfaces, complex state interactions, bridge dependencies, wrapped-asset risk, and monetary behavior that is difficult for ordinary operators to audit directly. Those designs may fit some use cases, but they also enlarge the attack surface and the validation burden.

Atho takes a narrower and more conservative path. It is built as a payment-oriented Layer 1 with deterministic validation, simple UTXO ownership semantics, proof-of-work ordering, explicit network separation, and post-quantum-aware signatures. The goal is not to maximize programmability at the base layer. The goal is to provide a settlement network that users, miners, wallet developers, exchanges, and auditors can reason about without ambiguity.

That philosophy appears throughout the implementation. Ownership is bound directly to canonical lock digests. Legacy lock forms are rejected. Full nodes validate emitted value against height-based subsidy rules. Block headers commit to founder-hash metadata, network identity, previous chain state, and canonical commitment roots. Storage is split between flat block archives and LMDB-backed indexed state. Wallets construct transactions locally, but they do not decide consensus truth. Consensus truth remains the job of full nodes.

2. Problem Statement

Atho exists because public settlement systems must solve several hard problems at once:

- they must transfer value without relying on a central operator to approve each spend
- they must expose rules that any independent node can validate deterministically
- they must remain useful for small payments and fee markets over long time horizons
- they must protect ownership through strong signatures and canonical transaction encoding
- they must isolate networks so that mainnet, testnet, regnet, and other modes cannot bleed into each other accidentally
- they must maintain a storage and synchronization model that is practical for operators rather than only for highly specialized infrastructure

The current Atho repository responds to those problems with explicit constants, strongly typed Rust modules, Falcon-512 authorization, SHA3-384 proof-of-work, Base56 visible addresses, canonical lock-digest ownership, deterministic transaction and block validation, and network-specific genesis anchors. The continuing engineering task is to preserve these rules in a form that is auditable, operator-readable, and consistent across node, wallet, miner, API, and explorer code.

3. Atho Design Principles

3.1. Security First

Atho prioritizes strict validation order, explicit ownership binding, fail-closed witness verification, deterministic block acceptance, and typed error handling. Invalid data should fail loudly and early.

3.2. Post-Quantum Awareness

Atho uses Falcon-512 for transaction authorization to reduce dependence on elliptic-curve signatures in its ownership model.

3.3. Proof-of-Work Fairness

Blocks are ordered through SHA3-384 proof-of-work. Miners compete by producing verifiable computational work rather than by receiving privileged issuance rights.

3.4. Predictability Through Explicit Monetary Rules

The current code enforces deterministic bootstrap-plus-tail issuance: the genesis block at height `0` pays no subsidy, the first ordinarily mineable block at height `1` begins a four-era reward schedule, and every later block after height `5,000,000` receives a fixed `0.50 ATHO` tail reward. All reward accounting stays in integer atoms, no floating-point math participates in consensus, and there is no hidden maximum-supply cutoff that turns otherwise valid tail-era blocks into invalid ones.

3.5. Payment Precision

Eight decimal places give Atho atom-level accounting: 1 ATHO equals 100,000,000 atoms, and all consensus amounts remain integer-only.

3.6. Simplicity Over Excessive Base-Layer Complexity

The base layer focuses on payments, ownership, block validation, and settlement. General-purpose virtual-machine complexity is intentionally not the center of the design.

3.7. Deterministic Operations

Nodes accept or reject the same canonical transaction and block bytes under the same consensus conditions. APIs, wallets, miners, and P2P paths are expected to feed into those same rules rather than bypass them.

4. System Overview

Atho is a proof-of-work payment network implemented in Rust and split across focused crates. `atho-core` defines canonical protocol objects, consensus constants, hashing, addresses, signing messages, and network identities. `atho-storage` validates chain objects in context and persists accepted state. `atho-crypto` owns Falcon-512 and hashing primitives. `atho-wallet` handles local wallet material and transaction construction. `atho-node` composes runtime behavior including mining, API, relay, and synchronization. `atho-qt` acts as a client interface rather than a second independent consensus engine.

The overall operating model is simple to state:

- wallets derive keys and build candidate transactions
- transactions are signed locally with Falcon-512
- peers and APIs feed raw bytes into strict decode and canonical validation
- the mempool stores only policy-valid, consensus-compatible transactions

- miners assemble candidate blocks from validated mempool entries
- full nodes independently validate blocks before committing state
- accepted blocks update the UTXO set, transaction indexes, and block metadata

4.1. Technical Overview of Current Code Parameters

Table 1 condenses the current code-grounded protocol parameters that frame the rest of the paper.

Table 1

Technical Overview of Current Code Parameters

Parameter	Current Code Value	Why It Matters
Consensus	Proof of Work	Orders blocks through locally verifiable work rather than delegated authority.
Proof-of-Work hash	SHA3-384	Provides a 384-bit mining digest with a modern sponge-based hash family.
Signature scheme	Falcon-512	Authorizes UTXO spends with a post-quantum-aware signature primitive.
Target block time	100 seconds	Sets the network's expected confirmation cadence and annual block count.
Block sizing model	SegWit-style vbytes and weight	Counts non-witness bytes at full weight while discounting witness bytes through virtual-size accounting.
Active launch block limit	1,000,000 bytes	Starting consensus block budget for epoch-zero and early-chain throughput.
Adaptive block-limit range	1,000,000 to 10,000,000 bytes	Lets blockspace expand or contract deterministically from prior chain usage.
Max block virtual size	10,000,000 vbytes	Primary throughput and fee-market limit used by miners and validators.
Max block weight	40,000,000 weight units	Equivalent to 10,000,000 vbytes under the 4:1 weight-to-vbyte scale.
Max serialized block size	10,000,000 raw bytes	Hard raw-byte ceiling for witness-heavy blocks and storage safety.
Normal transaction consensus spendability	1 confirmation	A normal transaction is confirmed and spendable once included in a valid block.
Wallet confirmation policy	User/app selected; default 3 confirmations	Wallets, merchants, and exchanges choose risk thresholds above the consensus floor.

Parameter	Current Code Value	Why It Matters
Coinbase maturity	100 blocks	Prevents immediate reuse of freshly mined subsidy outputs.
Decimals	8	Supports atom-level accounting with fixed E-8 precision.
Base unit	atom	Keeps all accounting in fixed integer units.
Atoms per ATHO	100,000,000	Prevents floating-point ambiguity in wallet and node accounting.
Genesis subsidy	0 ATHO at height `0`	Prevents a silent spendable premine from the anchor block.
Bootstrap issuance	18,750,000 ATHO	Fixed issuance delivered across four bootstrap eras before tail activation.
Permanent tail reward	0.50 ATHO from height `5,000,001`	Avoids a fee-only subsidy cliff and keeps issuance explicit after bootstrap.
Maximum supply	None	Valid tail-era blocks remain valid indefinitely under the same reward floor.
Ownership model	Canonical 32-byte lock digest	Binds spend authorization to a strict public-key-derived ownership commitment.
Address format	Base56	Gives users a visible network-aware address form over the canonical payment digest.
Validation model	Deterministic full-node validation	Keeps consensus truth in the node, not in miners, explorers, or wallets.

4.2. Design Goal of the System Overview

The purpose of the system overview is not merely architectural neatness. It is to make it clear where responsibility boundaries live. Wallet code is not allowed to define consensus truth. Explorer output is not allowed to redefine spendability. The network layer is not allowed to bypass local validation. Mining is not allowed to create a private shortcut around the rules.

4.3. Atho System Overview

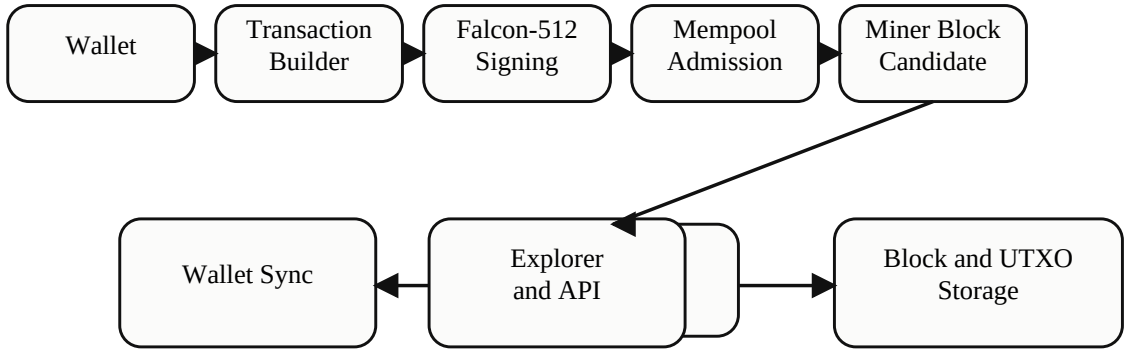


Figure 1. Atho System Overview

Atho persists accepted chain truth through a hybrid storage model. Canonical raw block bytes are archived in Bitcoin-style flat block data files, while LMDB stores block metadata, transaction archives, chainstate snapshots, UTXOs, peers, addresses, and peer health. APIs and explorer-like consumers can read from those state products, but they do not become consensus authorities. Figure 1 summarizes the movement from wallet transaction creation through storage, UTXO update, and downstream API or wallet synchronization.

5. Protocol Architecture

Atho is a Layer 1 proof-of-work blockchain with a public UTXO ledger. Its architectural boundary is deliberately modular. `atho-core` defines protocol objects, consensus constants, network identities, addresses, and signing messages. `atho-storage` performs contextual validation and durable state application. `atho-node` composes storage, mempool, mining, RPC/API, P2P, and runtime status. `atho-wallet` owns key derivation and wallet datafiles. `atho-qt` is a client of validated node state.

This separation matters because it reduces accidental consensus drift. If user-interface code changes, consensus behavior should not change with it. If explorer presentation changes, chain acceptance should remain unaffected. If mining or API code evolves, both paths should still route through the same validated state and object model.

The repository also keeps several versioning facts explicit rather than implied. The active P2P protocol version is `3`. The active storage schema version is `6`. The active ruleset version is `1`. The active block and transaction version are both `1`. A V2 ruleset placeholder exists in code with no activation height, which means future upgrade intent is visible without claiming that a second ruleset has already been deployed. The storage schema version is fixed in code rather than inferred from opaque local state.

5.1. Atho Node Architecture

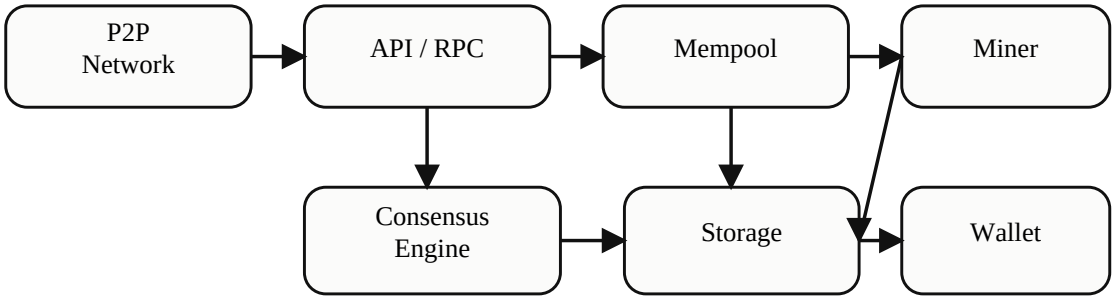


Figure 2. Atho Node Architecture

The live implementation keeps the network, API, mempool, consensus, storage, mining, and wallet interfaces separated enough that each can be inspected as its own responsibility boundary. P2P proposes data. API surfaces expose or submit data. The mempool tracks policy-valid pending transactions. Consensus validates blocks and transactions. Storage persists accepted truth. Wallet code manages keys and construction, but does not overrule the node.

6. Rust Implementation Strategy

Atho is implemented in Rust because the system benefits from explicit ownership semantics, strong typing, low-level performance control, and the ability to express consensus-critical behavior without garbage-collector pauses or unchecked memory mutation. That does not mean Rust automatically makes the protocol safe. It does mean the language gives the implementation a strong baseline for determinism, memory safety, and structured error handling when used carefully.

Consensus paths in Atho should continue to prefer domain types such as `Amount`, `TxId`, `BlockHash`, `Network`, and digest-width arrays rather than anonymous integers or unbounded strings when confusion is plausible. Recoverable failures should return typed errors. Network message handling should reject malformed input rather than panic. Serialization should stay canonical and testable. Unsafe shortcuts should remain excluded from consensus and storage paths.

6.1. Rust Safety Priorities in Practice

The current codebase already reflects several of those priorities:

- Falcon secret material uses zeroizing memory wrappers
- decoding functions reject malformed lengths and trailing bytes
- network identity is carried through typed enums and explicit visible prefixes
- consensus constants are centralized and unit-tested
- block headers and transactions use fixed byte encoders rather than ambiguous object serialization

Table 2 summarizes why Rust remains a strong fit for Atho’s core infrastructure.

Table 2

Why Rust Fits Atho Core Infrastructure

Requirement	Rust Advantage	Atho Use Case
Memory safety	Ownership and borrowing reduce broad classes of aliasing and use-after-free errors.	Consensus, storage, wallet state, and P2P parsing.
Explicit errors	`Result<T, E>` models rejection and recovery without exceptions.	Decode failures, validation rejection, API error surfaces, and wallet loading.
Performance	Native compilation and direct memory layout enable predictable hot-path behavior.	Proof-of-work loops, UTXO validation, mempool admission, and storage commits.
Concurrency discipline	Rust's type system makes data-race patterns harder to express incorrectly.	Parallel Falcon verification, background sync, and runtime status collection.
Modularity	Cargo workspaces and crates encourage narrow protocol boundaries.	`atho-core`, `atho-storage`, `atho-node`, `atho-wallet`, `atho-crypto`, and `atho-qt`.

Table 3 captures the practical Rust safety rules that matter most for consensus and storage code in the current Atho repository.

Table 3

Rust Design Requirements for Atho Core Infrastructure

Requirement	Reason	Enforcement Target
Strong domain types for amounts, heights, hashes, and addresses	Prevents unit confusion and wrong-context comparisons.	`atho-core` protocol types and validation APIs.
`Result`-based rejection for recoverable failures	Invalid network input is normal control flow, not exceptional control flow.	Decoders, validators, wallet import/export, and API handlers.
No `unsafe` in consensus-critical crates	Reduces the chance of hidden memory-unsafe behavior in the core rule path.	`atho-core`, `atho-storage`, `atho-node`, and `atho-crypto` boundaries.
Canonical byte encoding only	Prevents ambiguous hashes, txids, and signature messages.	Block, transaction, witness, and address encoding.
Malformed input must fail closed	Prevents peers, APIs, or wallets from smuggling partially valid state.	P2P codec, raw transaction decode, address decode, and witness parsing.
Upgrade points must stay explicit	Makes protocol changes reviewable before activation.	Ruleset scheduling, block/transaction versions, and storage schema versioning.

7. Cryptographic Design

Atho’s cryptographic surface is intentionally narrow. SHA3-384 is used for proof-of-work hashing, block and transaction identities, signing-message derivation, and checksum-related utilities where appropriate. Falcon-512 is used for spend authorization. The design goal is not to mix multiple signature families or complex script semantics into the same ownership surface. The goal is to keep ownership proofs and hashing commitments explicit.

Cryptographic design choices in Atho should be evaluated by the following questions:

- does the node reconstruct exactly the same signing message as the wallet
- does ownership bind to a canonical digest rather than loosely interpreted script bytes
- do malformed public keys and signatures fail before acceptance
- do network-specific contexts prevent cross-network misuse
- do hash commitments include the exact fields they are supposed to include and no hidden ambiguity

7.1. SHA3-384 in Atho

SHA3-384 produces a 384-bit digest and gives Atho a larger hashing margin than shorter digest sizes while preserving deterministic, machine-verifiable output for mining and block identification. It is standardized in NIST FIPS 202 [3]. In Atho, SHA3-384 is not a branding choice. It is the actual hash family used in consensus-facing code.

7.2. Post-Quantum Security Comparison

Table 4 positions Atho relative to common classical and post-quantum-aware ledger designs.

Table 4

Post-Quantum Security Comparison

System Type	Common Examples	Signature Type	Quantum Risk Profile	Atho Difference
Traditional Bitcoin-style systems	Bitcoin-like payment chains	ECDSA or Schnorr over elliptic curves	Long-term exposure in principle to Shor-style attacks against ECC if sufficiently capable quantum systems emerge.	Atho moves spend authorization to Falcon-512 rather than ECC.
Ethereum-style systems	Account-based smart-contract platforms	ECDSA over elliptic curves	Similar long-term ECC exposure, often with a larger smart-contract surface around key usage.	Atho narrows base-layer scope and uses a UTXO payment model with Falcon-512 witnesses.
Classical payment ledgers	Conventional digital asset and payment systems	Often centralized key management or classical asymmetric schemes	Security depends on institution and implementation model rather than public-node consensus validation.	Atho combines public-node validation, SHA3-384 proof-of-work, and post-quantum-aware signatures.
Post-quantum-aware systems	Research or newer signature-transition systems	Lattice-based or hash-based signatures	Designed to reduce dependence on elliptic-curve assumptions, but implementation maturity varies.	Atho places Falcon-512 directly in the live spend-authorization path.
Atho	Atho	Falcon-512	Designed for stronger long-term cryptographic resilience in transaction authorization without claiming permanent immunity to all future attacks.	Couples Falcon-512 authorization with SHA3-384 proof-of-work and deterministic UTXO validation.

7.3. Cryptographic Scope Discipline

The project should continue to resist unnecessary cryptographic surface growth. A payment chain benefits when signature rules, hash commitments, ownership digests, address derivation, and network separation are few enough to audit well.

8. Falcon-512 Implementation in Atho

Falcon-512 is the active transaction-signature profile in Atho. Public keys, secret keys, and signatures are represented through fixed-length wrappers in `atho-crypto`. The current implementation performs strict length checks on imported key bytes, rejects malformed witness material, and keeps secret-key material in zeroizing memory structures where possible.

In Atho, Falcon-512 is not treated as a loose interchangeable library choice. It is part of Atho's consensus profile. Nodes do not ask an external standards body at runtime what the signature format means; they follow the exact validation behavior frozen in the repository and accepted by the network. That is how Bitcoin-style consensus systems normally work as well: the chain's own consensus implementation is authoritative for what counts as valid. For Atho, that means the active Falcon wire format, public-key handling, signing digest construction, and verification behavior are defined by Atho's code, not by a future external library release changing underneath the network.

The role of Falcon-512 in Atho is narrow and important: it proves spend authorization for a specific transaction input under a specific signing context. It is not used for proof-of-work. It is not used as a general-purpose remote-authentication layer. It is used to authorize value transfer.

At the implementation level, the active wire and validation sizes are explicit: Falcon-512 public keys are fixed at `897` bytes, secret keys at `1,281` bytes, and signatures at `666` bytes. Those lengths are enforced at import and verification boundaries before the node attempts expensive witness verification. The node also rebuilds the canonical signing digest from transaction bytes and network context locally, rather than trusting wallet-supplied metadata about what was signed.

8.1. Consensus Profile and Standardization Boundary

The vendored `fn-dsa` code used by Atho explicitly warns that the FN-DSA standard is still in draft form and that the final published standard may differ in key encodings, message pre-hashing, and domain separation. NIST currently lists Falcon-derived FIPS 206 as in development [4]. That warning matters operationally, but it does not mean Atho is undefined. It means Atho should be described honestly: Atho uses a frozen Atho Falcon-512 consensus profile built around the vendored implementation present in the repository at release time.

That distinction has two practical consequences:

- Atho nodes remain internally interoperable as long as they run the same consensus code.
- Atho does not promise automatic future wire-level compatibility with whatever a later generic FN-DSA standards-compliant library may choose to encode or verify.

This is acceptable for a blockchain if it is explicit. A consensus system needs one authoritative validation profile, not a moving target. If a future FN-DSA standard release ends up differing from the vendored implementation Atho currently uses, Atho should not silently swap cryptographic behavior. Any migration should be introduced as a named, versioned protocol upgrade with activation rules, tests, and operator communication.

At the Atho wrapper level, signatures are not accepted as raw library-default objects alone. The node and wallet bind them to Atho-specific context. The signing path uses Atho domain labels, a `SHA3-384` prehash identifier, and Atho-built message digests. Transaction signatures commit to the transaction signing digest plus the active

network consensus ID and genesis hash. User-facing message proofs commit to the network, genesis hash, visible address, Falcon public key, and exact message bytes. This means Atho is not merely "using Falcon"; it is using Falcon within a narrower Atho-defined signing and verification envelope.

8.2. Vendored Implementation Provenance

The current repository vendors the Rust `fn-dsa` implementation as local path dependencies under `Falcon 512 rs/`. The top-level `fn-dsa` package manifest identifies the package as version `0.3.0`, lists the author as **Thomas Pornin**, and points to the upstream repository `https://github.com/pornin/rust-fn-dsa`. The vendored code is split across several local crates: `fn-dsa`, `fn-dsa-comm`, `fn-dsa-kgen`, `fn-dsa-sign`, and `fn-dsa-vrfy`. Atho does not claim authorship of that underlying Rust Falcon implementation. Atho vendors it, reviews it, and wraps it for Atho-specific consensus use.

The implementation notes inside the vendored code are also relevant to operators and auditors. The library describes itself as pure Rust and `no_std` compatible. It states that AVX2 acceleration is used automatically on x86 systems when available, while remaining compatible with CPUs that do not expose AVX2. It also states that signature generation uses hardware floating-point support on `x86_64` and ARMv8 targets when available for performance, while key generation and signature verification do not use floating-point operations. The vendored notes further explain that the key-generation path is a translation of the `ntrugen` code and that the signing path follows the original Falcon `sign_dyn` flow. These implementation details matter because they help explain what Atho is actually shipping: not a vague "post-quantum module," but a concrete vendored Rust implementation with stated performance and portability characteristics.

On top of that vendored implementation, Atho adds its own wrapper layer in `atho-crypto`. That wrapper is responsible for fixed-width key and signature types, zeroizing secret handling, deterministic seed expansion for wallet derivation, Atho-specific signing domains, SHA3-384 prehash identifiers, and fail-closed verification behavior at wallet and node boundaries. In other words, the underlying Falcon math implementation is vendored, but the consensus envelope around it is Atho-specific.

8.3. Signature Verification and Rejection Behavior

The consensus consequences of witness verification are more important than the abstract signature primitive. The node must reject:

- malformed public keys
- malformed signatures
- wrong-message signatures
- signatures attached to the wrong UTXO
- signatures whose public key does not map to the expected ownership digest
- missing witness components
- replay attempts across incompatible network or ownership contexts

Table 5 summarizes the expected behavior for common Falcon-512 failure cases.

Table 5

Falcon-512 Validation Failure Cases

Failure Case	Detection Method	Expected Result
Malformed public key	Length check and decode failure in the verification path	Reject as invalid witness or key material.
Malformed signature	Signature length check or verifier failure	Reject before mempool or block acceptance.
Wrong signing message	Node rebuilds the canonical signing digest and verifies against witness	Reject as invalid witness.
Changed transaction output	Base transaction bytes change the `txid` and signing context	Existing signature no longer verifies.
Wrong UTXO ownership	Lock digest derived from public key does not match stored canonical lock	Reject as ownership mismatch.
Missing signature or key	Witness payload lacks required fields	Reject as malformed witness.

9. Transaction Model

Atho transactions consume existing UTXOs and create new UTXOs. Ownership is not inferred from a broad script interpreter. Instead, spendability flows through canonical transaction serialization, explicit input references, output value accounting, and Falcon-512 witness authorization tied to a 32-byte canonical lock digest.

The transaction body is serialized deterministically. Transaction IDs derive from canonical bytes. Witnesses are stored separately from the base transaction identity so the node can preserve a stable base transaction digest while still validating the witness payload needed to authorize spends.

The current implementation also applies transaction-level resource and anti-spam policy with explicit constants. Standard transactions are bounded at `250,000` raw bytes and `250,000` vbytes. Required transaction fee is `max(600 atoms, 1 atom per serialized byte)`. Dust-like outputs below `100` atoms are rejected, and standard transactions are capped at `64` outputs and `1,024` inputs. Wallets can pay only that minimum, use a suggested priority fee rate from a lightweight mempool estimator, or choose a higher optional tip rate. Some API fields retain vbyte-oriented names for compatibility and priority guidance, but final transaction acceptance depends on deterministic structural validation, dust rules, witness correctness, the required fee floor, and valid TX-PoW. These limits are policy-facing, but they shape the operational transaction model that wallets, APIs, and miners actually use.

Normal non-coinbase transactions also carry wallet transaction proof-of-work in the current code. The wallet signs the transaction first, then derives a `SHA3-256` anti-spam preimage under the `ATHO_TX_POW_V1` domain, network consensus ID, and genesis hash, and solves for a nonce that satisfies the required difficulty bits. That per-transaction PoW is deterministic on every network and scales from `10` to `16` bits based on transaction size, input count, and output count. The schedule starts at `10` bits; adds `2` bits at each vsize threshold above `500`, `1,000`, and `2,000` vbytes; adds `1` bit above `4` and `16` inputs; adds `1` bit above `2`, `8`, and `32` outputs; and clamps the result to the inclusive `10`-to-`16` range. A transaction must declare the exact computed target. The design goal

is to add sender-side cost for abusive traffic without weakening block-level consensus proof-of-work, while keeping ordinary wallet sends responsive on lower-powered devices. TX-PoW is a pass/fail anti-spam gate, not a replacement fee market and not a bid system where extra work changes consensus validity.

9.1. Transaction Signing and Verification Flow

Figure 3. Atho Transaction Signing and Verification Flow

Wallet code selects spendable UTXOs, constructs a canonical transaction body, derives the signing digest, signs it with Falcon-512, and broadcasts the result. The node then rebuilds that digest locally and verifies it against the witness material. That reconstruction step is essential. Consensus cannot trust a wallet's claim about what it signed.

9.2. Transaction Lifecycle

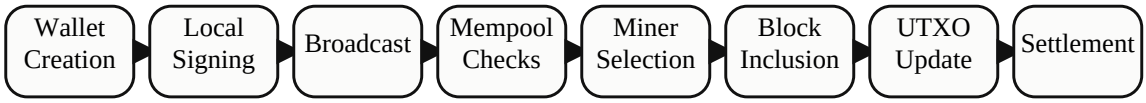


Figure 4. Transaction Lifecycle

The normal lifecycle is wallet construction, local signing, raw-byte relay, mempool validation, miner selection, block inclusion, full block validation, UTXO update, and confirmations. Each step has a different responsibility, but none is allowed to override consensus validity.

10. UTXO and Accounting Rules

The UTXO set is the live accounting surface of Atho. Every accepted spend removes previously unspent outputs and creates new ones. Correctness therefore depends on four properties:

- referenced outpoints must exist and be unspent
- the spending witness must authorize the exact UTXO being consumed
- the sum of created value must not exceed allowed input value plus valid subsidy where applicable
- duplicate spends must be rejected both within a transaction and within a block

The current repository uses canonical lock digests rather than permissive legacy script forms. Noncanonical locking data is rejected. Oversized or malformed witness data is rejected. Coinbase outputs remain locked behind

a maturity threshold before they are spendable.

Table 6 summarizes the current UTXO validation rules that matter most for operators and integrators.

Table 6

UTXO Validation Rules

Rule	Current Behavior	Why It Matters
Normal transaction spendability	1 confirmation at consensus	A normal UTXO can be spent once mined into the active best chain.
Wallet/app confirmation policy	User/app selected	Wallets, merchants, and exchanges can require 0, 1, 3, 6, 20, or more confirmations based on risk.
Coinbase spendability	100 confirmations	Prevents immediate reuse of freshly mined subsidy outputs.
Lock format	Canonical 32-byte ownership digest	Prevents ambiguous or legacy anyone-can-spend behavior.
Duplicate inputs	Rejected	Stops same-transaction double spends.
Missing UTXO	Rejected	Prevents phantom value creation.
Ownership mismatch	Rejected	Ensures the witness key matches the exact lock being spent.

11. Block Structure

Each Atho block contains a canonical header plus a transaction list headed by exactly one coinbase transaction at index 0. Non-coinbase transactions in the same block cannot use coinbase shape. The header commits to the network ID, version, previous block hash, commitment roots, timestamp, difficulty bits, nonce, and founder-hash metadata fields. Those founder hashes are part of canonical header bytes in the current implementation and therefore part of block identity.

Block validation is not only about proof-of-work. The block also has to satisfy structural rules, canonical transaction decoding, coinbase placement rules, duplicate-spend rejection, UTXO validity, and monetary correctness. Valid proof-of-work alone does not rescue an invalid block.

The coinbase transaction must encode the exact block height in its canonical `lock_time` field. Because that field is committed by the base transaction bytes, it makes the coinbase txid height-specific and prevents a miner from replaying an older coinbase identifier after its original output has been spent. Nodes enforce the rule during ordinary block connection, branch replay, snapshot import, reindexing, and reorganization validation. A stale-height coinbase is rejected before UTXO mutation. Same-block duplicate txids are also rejected independently. This is Atho's defense against the historical class of BIP30-style duplicate-transaction ambiguity. The current `u32` commitment supports heights through `4,294,967,295`, more than 13,000 years at the target 100-second cadence, and validation fails closed above that format boundary.

The frozen commitment-tree algorithm duplicates the final hash of an odd layer. Validation preserves those exact roots while separately detecting equal real siblings and duplicate base txids. This prevents a duplicate-last

body from sharing a valid header's roots and poisoning local invalidity state. A same-txid transaction carrying different witness or TX-PoW data is also rejected: its full identity differs, but it cannot replace the base identity in the mempool or appear beside that identity in a block.

The header structure matters because it is the smallest object whose bytes affect chain ordering. Atho's canonical header contains: block version, network ID, height, previous block hash, merkle root, witness root, founder-hash `SHA3-384`, founder-hash `SHA3-512`, timestamp, target bytes, and nonce. Nodes hash those bytes deterministically and compare the resulting digest against the target. Any disagreement about field order, digest width, or included metadata would create immediate consensus breakage.

The live implementation also makes block resource limits explicit: `10,000,000` vbytes, `10,000,000` raw serialized bytes, and `40,000,000` weight units. Blocks carry total-fee and miner-fee summary fields for storage and API reporting, but those summaries are not part of the canonical header commitment and are not trusted for consensus. During block connection, chainstate derives both values from the already validated coinbase total minus the active subsidy before persisting the block. The consensus reward rule remains the exact checked sum of subsidy and included transaction fees.

11.1. Block Size, SegWit-Style Weight, and TPS

Atho uses SegWit-style block accounting rather than a single raw-byte block limit. The validator calculates block weight as:

```
block_weight = (base_bytes * 3) + raw_serialized_bytes
block_vbytes = ceil(block_weight / 4)
```

Because `raw_serialized_bytes = base_bytes + witness_bytes`, this is equivalent to counting base transaction data at 4 weight units per byte and witness data at 1 weight unit per byte. In practical terms, non-witness transaction data consumes the scarce vbyte budget more heavily, while Falcon witness material is still committed, serialized, bounded, and validated under the raw-byte and witness-root rules.

The active block limits are:

- adaptive launch block limit: 1,000,000 bytes
- maximum virtual block size: 10,000,000 vbytes
- maximum block weight: 40,000,000 weight units
- maximum raw serialized block size: 10,000,000 bytes
- target block time: 100 seconds

Estimated transaction throughput is therefore a function of average serialized transaction size and vsize; the active adaptive raw-byte limit is the binding limit for byte-heavy blocks:

```
approx_tx_per_block = floor(active_block_limit_bytes / average_tx_bytes)
approx_tps = approx_tx_per_block / 100
```

Approximate L1 capacity using a 590-byte average transaction:

Active block limit	Approx tx/block	Approx TPS
1 MB	1,694	16.9
2 MB	3,389	33.9
5 MB	8,474	84.7
10 MB	16,949	169

These are capacity estimates from consensus sizing constants, not a promise that every deployed node will sustain those numbers under all conditions. Real throughput also depends on average input count, output count, witness size, mempool load, peer bandwidth, block propagation, disk performance, Falcon verification throughput, and block packing. The relevant design fact is that Atho launches at a 1,000,000-byte active block limit every 100 seconds and can grow toward a 10,000,000-byte ceiling only after sustained demand.

11.2. Adaptive Block-Limit Epoch Flow

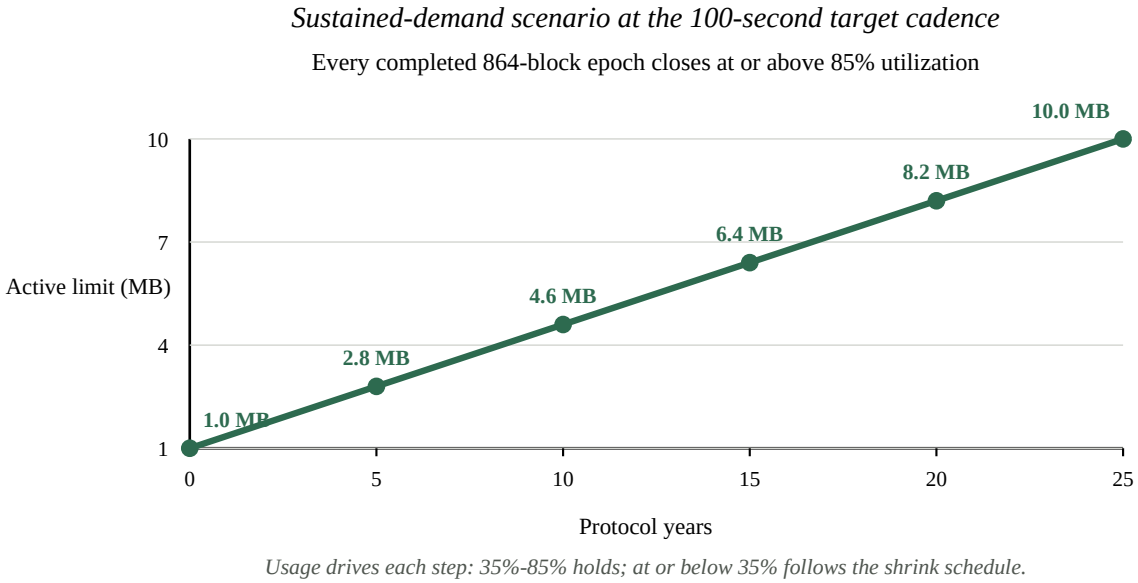


Figure 5. Adaptive Block-Limit Growth Under Sustained Demand

The adaptive block limit is a chain-derived next-block rule, not an operator knob. Each accepted canonical block contributes its raw consensus size to the current `864`-block epoch. When that epoch closes, the node compares cumulative usage against the epoch capacity implied by the current limit. Usage at or above `85%` advances the limit by one deterministic growth bucket across `9,125` epochs, which corresponds to about 25 protocol years under sustained high usage. Usage at or below `35%` retreats by one deterministic shrink bucket across `1,825` epochs, or about 5 protocol years under sustained low usage. Anything between those thresholds holds the limit steady. The result is always clamped back into the `1,000,000`-to-`10,000,000` byte range.

Figure 5 shows the upper growth path when every completed epoch remains at or above the expansion threshold. It is an explanatory scenario, not an automatic time schedule or capacity forecast: real chain usage can make the limit grow, hold, or shrink at each epoch boundary.

The adaptive limit does not replace the hard block caps. Every candidate block must still satisfy the absolute raw-byte, vbyte, and weight ceilings. The adaptive rule simply decides the smaller active raw-byte budget for the next direct canonical tip extension.

```
Adaptive Block-Limit Epoch Flow

  current canonical block limit L
              |
              v
  add each accepted canonical block's raw consensus
  bytes into the current 864-block epoch total
              |
              v
  epoch closed?
      |
      +-- no  -> next direct tip extension still uses L
      |
      +-- yes
              |
              v
  usage >= 85% of (L * 864)?
      |
      +-- yes -> map L into 9,125 growth buckets,
                 advance one bucket, clamp at 10,000,000
      |
      +-- no
              |
              v
  usage <= 35% of (L * 864)?
      |
      +-- yes -> map L into 1,825 shrink buckets,
                 retreat one bucket, clamp at 1,000,000
      |
      +-- no  -> next limit = L
              |
              v
  persist new adaptive state and keep enforcing
  hard caps: 10,000,000 raw bytes, 10,000,000 vbytes,
  40,000,000 weight
```

12. Proof-of-Work and Mining

Atho uses SHA3-384 proof-of-work. Miners search the nonce space for a header hash below the active target. The node’s mining path is not supposed to create a separate private notion of validity. Candidate blocks must still pass the same validation rules that apply to blocks received from peers.

The active code uses a 100-second target block time and a height-based subsidy schedule. Mining therefore combines two tasks:

- ordering pending valid transactions into a candidate block
- solving for valid proof-of-work under the current target

Difficulty retargeting is also explicit rather than inferred. The current profile retargets every block using a `17`-block averaging window, an `11`-block median-time window, a damping factor of `4`, a maximum upward adjustment of `16%`, and a maximum downward adjustment of `32%`. This makes difficulty movement responsive enough for shorter block times while still bounding abrupt oscillation.

Fork choice is based on cumulative proof of work, not block count alone. For a 384-bit target `T`, the implementation assigns each block the following integer work contribution and sums it across the branch:

```
block_work(T) = floor((2^384 - 1 - T) / (T + 1)) + 1
branch_work   = sum(block_work(T) for each block in the branch)
```

The valid branch with greater accumulated work is preferred. Height is used only after equal chainwork, followed by a deterministic tip-hash comparison. Testnet also permits a minimum-difficulty reset after a `600`-second stall; that network-specific recovery rule does not apply to mainnet.

12.1. Mining Responsibilities

Mining must:

- build a coinbase transaction that does not overpay subsidy plus optional tips
- include only transactions that remain valid under current UTXO state and policy
- respect canonical block serialization
- preserve founder-hash header metadata
- submit solved blocks through the same full-validation path used for externally sourced blocks

Table 7 summarizes the practical mining responsibilities exposed by the current codebase.

Table 7

Mining Components and Responsibilities

Component	Current Role	Validation Constraint
Block template builder	Selects candidate mempool transactions and constructs a coinbase.	Must use transactions that still pass node validation at template time.
Coinbase builder	Pays subsidy plus allowed optional tips to the configured output.	Must never overpay the height-based reward plus tips.
Merkle and witness commitment logic	Commits ordered transaction and witness data into the header.	Must match the validator’s recomputation exactly.
Proof-of-work loop	Searches nonce space for a header hash below the current target.	Valid proof-of-work cannot rescue an invalid block body.
Local block submission path	Returns solved blocks to the node.	Must re-enter the same full block validation path as peer-sourced blocks.
GPU/native acceleration	Optimizes hash-search throughput where supported.	Must not alter canonical header bytes, target rules, or submission behavior.

The latest mining path no longer uses public deterministic reward keys for value-bearing networks. Mainnet and testnet candidate construction require an explicit configured Atho reward address through `ATHO_MINING_REWARD_ADDRESS` or `atho.conf`, and the address must decode for the node’s active network. Deterministic development reward defaults are limited to regnet and prunetest.

13. Monetary Policy and Emissions

The current Atho codebase enforces deterministic bootstrap-plus-tail issuance. Every valid reward-bearing block recomputes its subsidy from canonical height ranges, keeps all accounting in integer atoms, and may add included transaction-fee revenue on top of that subsidy. The monetary policy is intentionally simple to state, simple to audit, and simple for every full node to recompute from integer constants alone.

13.1. Atho Emission Policy and Long-Term Miner Security

Atho now uses a bootstrap-plus-tail proof-of-work emission model rather than a finite hard-cap halving schedule. The genesis block at height `0` is a zero-emission anchor. The first ordinarily mineable block is height `1`. Subsidy then follows four deterministic bootstrap eras of `1,250,000` blocks each:

- heights `1 ..= 1,250,000`: `8 ATHO`
- heights `1,250,001 ..= 2,500,000`: `4 ATHO`
- heights `2,500,001 ..= 3,750,000`: `2 ATHO`
- heights `3,750,001 ..= 5,000,000`: `1 ATHO`
- heights `>= 5,000,001`: `0.50 ATHO` forever

That schedule yields exactly `18,750,000 ATHO` of bootstrap issuance before the tail begins. After that point, Atho keeps a permanent `0.50 ATHO` base reward, equal to `50,000,000` atoms, on every later block. There is no premine and no maximum-supply consensus rule that invalidates future tail-emission blocks.

This is not just descriptive language. It is the live consensus rule set enforced by the repository. Nodes compute rewards from canonical height ranges, pay height `0` exactly zero, and clamp all later tail-era rewards to the same `50,000,000`-atom floor instead of repeatedly right-shifting toward zero. The result is intentionally different from both a fee-only end state and a capped schedule with a special terminal-dust closeout.

The security thesis is straightforward. Bootstrap rewards provide a predictable issuance runway while the network is young. The permanent tail reward avoids a sudden cliff where subsidy disappears and miners must rely solely on fee demand at one discrete future height. Transaction fees still matter because every non-coinbase transaction must pay a consensus minimum fee and may optionally add extra tip above that floor, but miner security is not modeled as a pure all-at-once transition into fee-only mining.

13.2. Design Reasoning

The updated design aims to keep the monetary rules auditable without creating a hidden future cutoff. Four discrete bootstrap eras are easier to communicate than an indefinite right-shift series, and the permanent tail reward is easier to reason about than a schedule that asymptotically approaches zero and then needs a special cap-closing exception.

Several design choices matter here:

- the genesis block does not silently create a spendable reward
- the first ordinarily mineable block is explicitly height `1`
- every subsidy amount is integer-only in atoms
- height boundaries are inclusive and documented in tests
- no compatibility alias or fallback maximum-supply field is treated as live consensus truth

- explorer and API surfaces distinguish scheduled issuance from circulating, burned, and immature balances instead of conflating them

The result is that a node, miner, wallet, or exchange can reason about the active subsidy from height alone. A bootstrap-era node can estimate its current-year issuance from the active era. A tail-era node can compute the steady-state issuance floor immediately. None of those components need floating-point arithmetic, heuristic era inference, or a special-case terminal mode that changes the network's monetary semantics after a one-time cutoff event.

13.3. Exact Consensus Rules

Table 8 summarizes the frozen monetary-policy constants used by the current implementation.

Table 8

Monetary Policy Constants and Current Consensus Values

Constant	Current Value	Operational Meaning
Target block time	100 seconds	36 blocks per hour, 864 blocks per day, and 315,360 blocks per year.
Decimals	8	1 ATHO is represented as 100,000,000 atoms.
Smallest unit	1 atom	Equal to 0.00000001 ATHO.
Genesis subsidy	0 ATHO at height `0`	The anchor block exists without silently minting a spendable premine.
First bootstrap reward	8 ATHO for heights `1 ..= 1,250,000`	First miner-emission era, equal to `800,000,000` atoms per block.
Second bootstrap reward	4 ATHO for heights `1,250,001 ..= 2,500,000`	Second reward era, equal to `400,000,000` atoms per block.
Third bootstrap reward	2 ATHO for heights `2,500,001 ..= 3,750,000`	Third reward era, equal to `200,000,000` atoms per block.
Fourth bootstrap reward	1 ATHO for heights `3,750,001 ..= 5,000,000`	Final bootstrap era, equal to `100,000,000` atoms per block.
Tail reward	0.50 ATHO from height `5,000,001` onward	Permanent subsidy floor, equal to `50,000,000` atoms per block.
Bootstrap interval	1,250,000 blocks	Exact length of each bootstrap era.
Bootstrap issuance	18,750,000 ATHO	Total scheduled issuance before tail activation.
Maximum supply	None	Tail-era blocks remain valid indefinitely if the rest of consensus passes.
First-era annualized base issuance	2,522,880 ATHO	Year-1 issuance implied by `8 ATHO` times `315,360` blocks.
Tail annualized base issuance	157,680 ATHO	Steady-state issuance implied by `0.50 ATHO` times `315,360` blocks.
Premine	None	There is no privileged genesis treasury or emission carve-out.

Constant	Current Value	Operational Meaning
Reward-step model	Four bootstrap steps plus permanent tail	Avoids zero-subsidy cliffs and special terminal-dust closeout logic.
Required transaction fee	`max(600 atoms, 1 atom per serialized byte)`	Floor-only transactions are consensus-valid without an extra tip.
Suggested optional tip rate	1 atom/vbyte	Wallet and mempool guidance during congestion.
Dust threshold	100 atoms	Equal to 0.00000100 ATHO.
TX-PoW difficulty	10 to 16 bits	Sender-side anti-spam work scales with transaction size and shape.
Coinbase maturity	100 blocks	Earliest block age before coinbase outputs become spendable.
Normal transaction spendability	1 confirmation	Consensus floor for normal UTXO spendability.
Wallet confirmation policy	User/app selected; default 3 confirmations	Applications choose risk thresholds above the consensus floor.

The exact consensus rule is straightforward:

- 8 decimals
- 1 ATHO = 100,000,000 atoms
- genesis subsidy at height `0` = `0`
- heights `1 ..= 1,250,000` pay `800,000,000` atoms
- heights `1,250,001 ..= 2,500,000` pay `400,000,000` atoms
- heights `2,500,001 ..= 3,750,000` pay `200,000,000` atoms
- heights `3,750,001 ..= 5,000,000` pay `100,000,000` atoms
- heights `>= 5,000,001` pay `50,000,000` atoms forever
- target block time = 100 seconds
- blocks per day = 864
- blocks per year = 315,360
- bootstrap interval = 1,250,000 blocks
- bootstrap issuance through height `5,000,000` = `18,750,000 ATHO`
- first-era annualized base issuance = `2,522,880 ATHO`
- tail annualized base issuance = `157,680 ATHO`
- no maximum-money rule invalidates later tail-era blocks

- allowed coinbase value = active subsidy + total included transaction fees
- total included transaction fee must be at least `max(600 atoms, 1 atom per serialized byte)` for every non-coinbase transaction
- any fee paid above that floor is an optional user-selected tip and remains miner revenue if the transaction is included
- no premine

Every full node validates these rules locally. Coinbase reward calculation is deterministic across all nodes because the implementation uses integer atom units only. There is no floating-point reward path in consensus.

Subsidy calculation flow:

```
candidate block height h
        |
        v
h == 0?
  |
  +-- yes -> subsidy = 0
  |
  +-- no
        |
        v
era = (h - 1) / 1,250,000
        |
        v
era == 0 ? subsidy = 800,000,000
era == 1 ? subsidy = 400,000,000
era == 2 ? subsidy = 200,000,000
era == 3 ? subsidy = 100,000,000
otherwise subsidy = 50,000,000
        |
        v
coinbase may claim subsidy plus included transaction fees only
```

13.4. Numeric Bounds and the u64 Amount Ceiling

The codebase uses `u64` for individual atom-denominated amounts such as output values, optional-tip totals, and per-transaction balances, while long-horizon aggregate issuance helpers use `u128`. That distinction matters.

The theoretical maximum value of one `u64` amount field is `18,446,744,073,709,551,615` atoms, which equals `184,467,440,737.09551615 ATHO` at 8 decimals. That is far above any individual bootstrap reward, tail reward, transaction output, or fee figure used by the live consensus constants.

That means the live consensus schedule never gets close to amount-field exhaustion. The largest ordinary block reward is `800,000,000` atoms, the permanent tail reward is `50,000,000` atoms, transaction and coinbase arithmetic uses checked integer math, and cumulative issuance helpers use `u128`. The important property is not that integers accidentally imply a hidden cap. The important property is that the implementation has enough numeric headroom to represent both the bootstrap schedule and indefinite tail-era reporting without overflow over the full `u64` block-height domain.

This distinction matters for audits. Reasonable integer ceilings still exist so the software can reject impossible values and avoid arithmetic corruption, but those guards are not consensus substitutes for a maximum-money rule. Tail-emission blocks remain valid because the monetary policy says they remain valid, not because the code “happens not to overflow yet.”

13.5. Cadence-Derived Issuance Through Year 135

Atho target-cadence emission simulation through year 135

8/4/2/1 ATHO bootstrap eras, then 0.50 ATHO permanent tail

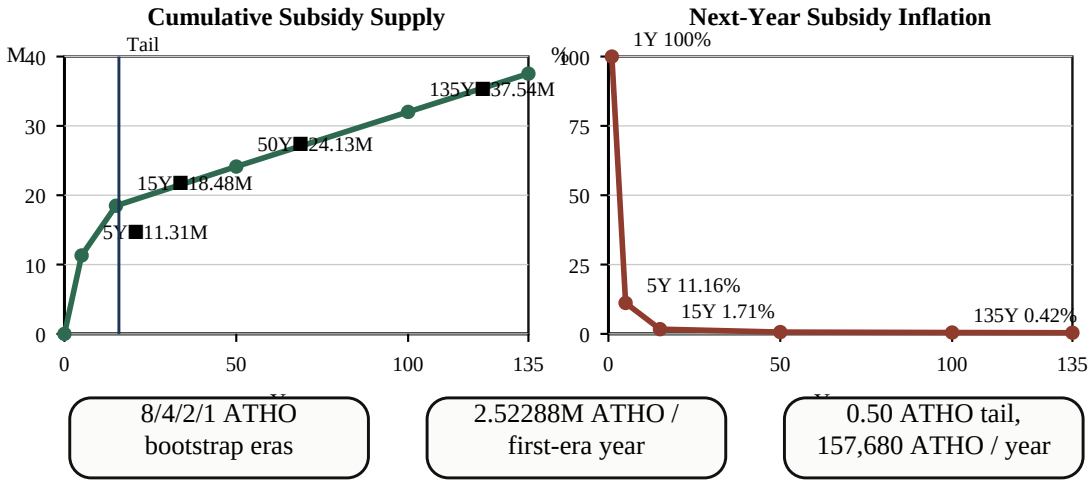


Figure 6. Atho Emission Simulation Through Year 135

Because subsidy is height-based, long-horizon issuance is derived from the base reward, the halving interval, and the target cadence. At the active `100`-second target:

- `36` blocks per hour
- `864` blocks per day
- `6,048` blocks per 7-day week
- `315,360` blocks per nominal 365-day year
- tail activation after `5,000,000` blocks, or about `15.85` target years
- first-era issuance of `6,912 ATHO` per day and `2,522,880 ATHO` per year
- tail-era issuance of `432 ATHO` per day and `157,680 ATHO` per year

Those cadence figures imply a bootstrap phase that declines in four steps and then hands off to a flat absolute tail issuance floor. The schedule is still easy to audit from constants alone. Bootstrap supply reaches `18,750,000 ATHO` at height `5,000,000`, and every later year adds another `157,680 ATHO` of scheduled base issuance before fees and optional user-selected tip.

These are not market forecasts, treasury promises, or price targets. They are the target-cadence arithmetic that every full node can recompute from consensus constants alone. Actual wall-clock production can vary around target because proof-of-work is probabilistic, but the policy expression itself is deterministic and auditable.

Figure 6 turns the same bootstrap-plus-tail schedule into two visual views. The cumulative-supply panel rises quickly in the first era, keeps climbing through the later bootstrap eras, then continues linearly after tail activation. The inflation panel steps down across the four bootstrap reward reductions and then continues declining more gradually because the tail reward is fixed in absolute terms while the cumulative supply base keeps growing.

The figure should be read with four explicit assumptions:

- target cadence averages `100` seconds over the long run

- only base subsidy is modeled; optional tips are excluded
- the schedule is shown in nominal 365-day years
- the chart is deterministic arithmetic, not a price model or adoption forecast

That distinction matters. The projection does not end at a hard cap because the tail reward is permanent. What changes over time is not whether issuance stops, but how large annualized issuance is relative to the existing scheduled supply base.

Table 9 gives selected nominal-year anchors. Year `1` remains a useful early anchor: `315,360` expected blocks, `2,522,880 ATHO` of cumulative scheduled issuance, and `100%` next-year subsidy inflation against the year-1 scheduled supply base. Between years `15` and `20`, the schedule crosses from the final bootstrap era into the tail phase.

Table 9

Five-Year Emission Schedule Through Year 135

Time Horizon	Expected Blocks	Cumulative Subsidy Issued	Next-Year Issuance as % of Existing Subsidy Supply
Year 1	315,360	2,522,880 ATHO	100.000000%
Year 5	1,576,800	11,307,200 ATHO	11.156078%
Year 10	3,153,600	16,307,200 ATHO	3.867739%
Year 15	4,730,400	18,480,400 ATHO	1.706457%
Year 20	6,307,200	19,403,600 ATHO	0.812633%
Year 25	7,884,000	20,192,000 ATHO	0.780903%
Year 50	15,768,000	24,134,000 ATHO	0.653352%
Year 100	31,536,000	32,018,000 ATHO	0.492473%
Year 135	42,573,600	37,536,800 ATHO	0.420068%
Year 150	47,304,000	39,902,000 ATHO	0.395168%
Year 175	55,188,000	43,844,000 ATHO	0.359639%
Year 200	63,072,000	47,786,000 ATHO	0.329971%

Several operational implications follow from that table.

First, the policy is easy to audit over very long periods because every era is derived from one interval and one explicit reward floor. A node, explorer, exchange back office, or external auditor can recompute the current reward, the tail activation point, and any selected long-horizon supply milestone from constants alone.

Second, the bootstrap phase is intentionally finite even though the overall emission schedule is not. Height `5,000,000` marks the end of the stepped bootstrap eras and height `5,000,001` marks the first permanent tail block. That gives operators a clean phase boundary without describing that boundary as a hard cap.

Third, the percentage effect of new subsidy declines over time but does not hit exactly `0%` as long as the tail reward exists. That decline is driven first by bootstrap reward reductions and then by supply growth against a fixed annualized tail emission.

The 200-year row makes the long-run schedule concrete without implying a hidden cap. At nominal target cadence, Atho reaches `47,786,000 ATHO` of cumulative scheduled subsidy by year `200`. That total consists of the fixed bootstrap issuance of `18,750,000 ATHO` plus `29,036,000 ATHO` from tail-era blocks after height `5,000,000`. The next nominal year would still add `157,680 ATHO`, but that same fixed annualized tail issuance would represent only `0.329971%` of the existing scheduled subsidy base.

13.6. Inflation Decline Under Permanent Tail Emission

The inflation behavior is already encoded in Table 9, so the important job of this section is interpretation rather than repetition.

The key observation is that Atho's annualized subsidy rate falls in two ways. First, the absolute reward steps down from `8` to `4`, then `2`, then `1`, then `0.50 ATHO`. Second, the same annualized issuance becomes a smaller fraction of an expanding cumulative supply base. At year `5`, the next-year annualized subsidy is about `11.156078%` of existing scheduled supply. At year `20`, after tail activation, that figure is about `0.812633%`. At year `100` it is about `0.492473%`, at year `200` it is about `0.329971%`, and by year `500` it falls to about `0.165822%` while supply is still growing linearly.

That is why the schedule matters. It is not a promise about real wall-clock timestamps, but it is deterministic target-cadence arithmetic. A slower or faster actual block stream changes elapsed wall time, not the height-based subsidy sequence that full nodes enforce.

13.7. Miner Security

Miners receive predictable block rewards under a deterministic bootstrap schedule and then a fixed permanent tail reward. A miner can model the subsidy side of future revenue from the current height, the `1,250,000`-block era interval, and the known tail floor at height `5,000,001`.

This matters because proof-of-work security is a live market for hash power. Atho chooses a strong first-era subsidy runway, deterministic reductions after that, and then a permanent baseline reward rather than a one-time jump into a fee-only world. That does not guarantee future profitability or absolute security. It does mean the monetary policy avoids a discrete “subsidy ends here” cliff.

Transaction fees still matter in Atho. Every non-coinbase transaction must pay at least the consensus fee floor, and users may choose to pay more than that minimum when they want stronger inclusion priority. Miners therefore earn the active subsidy plus included fees during bootstrap and the tail reward plus included fees afterward. The tail reward is a base floor, not a claim that fees stop being important.

13.8. Optional Tips and TX-PoW

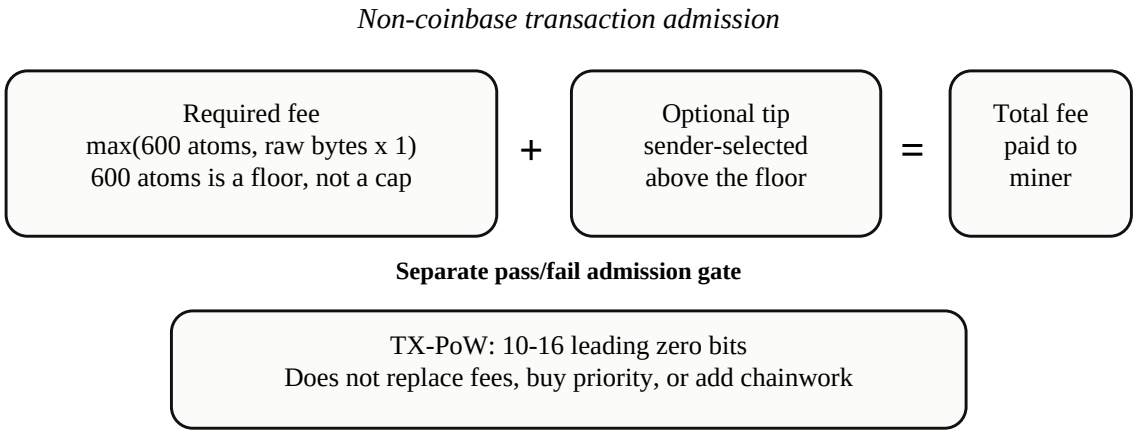


Figure 7. Transaction Admission Cost Components

This emission model sits beside a fee-floor transaction policy rather than replacing it. Consensus requires every non-coinbase transaction to pay at least `max(600 atoms, 1 atom per serialized byte)`. Every such transaction must also satisfy deterministic TX-PoW, and users may optionally add fee above that floor when they want stronger inclusion priority during congestion.

Figure 7 separates those components: the enforced fee floor and optional tip form miner fee revenue, while TX-PoW remains an independent pass/fail anti-spam requirement.

The important accounting distinction is this:

- the full fee collected by a valid included transaction is miner revenue
- the minimum required part of that fee is the consensus floor
- the amount above that floor is the optional user-selected tip

Atho does not introduce fee burning in this update. Included transaction fees remain miner revenue during bootstrap and tail eras alike.

Tip accounting flow:

```
valid transaction
      |
      v
sum input atoms - sum output atoms
      |
      v
minimum required fee = max(600 atoms, 1 atom per serialized byte)
      |
      v
total fee < required fee?
  |
  +-- yes -> reject transaction
  |
  +-- no
      |
      v
optional tip = total fee - required fee
      |
      v
block template sums all included transaction fees
```

```
      |
      v
allowed coinbase value = active subsidy + total included fees
  |
validation rejects any coinbase overpayment, including 1 atom too much
```

The 8-decimal display model still keeps tiny units understandable in ordinary wallet and exchange interfaces. One atom is `0.00000001 ATHO`. The minimum dust threshold is `100` atoms, or `0.00000100 ATHO`. The fee floor starts at `600` atoms and also scales by `1` atom per serialized byte, while a suggested optional tip rate of `1` atom per vbyte remains visible enough for UI guidance without changing the backend's integer-only accounting model.

TX-PoW remains the companion anti-spam mechanism. The wallet signs the transaction first, then solves a deterministic sender-side work target that now scales from `10` to `16` bits based on transaction size and shape. Extra TX-PoW does not buy higher priority by itself. Priority still comes from the fee a sender chooses to pay above the enforced floor, from miner policy, and from available block space under the active adaptive limit.

13.9. Comparison to Halving Models

Atho uses halvings, but it does not use Bitcoin's exact subsidy interval, monetary cap, signature system, or block cadence. That is not a claim that Bitcoin's model is invalid. It is a statement that Atho is optimizing for a different long-term security posture.

Traditional halving models are often paired with an eventual zero-subsidy endpoint. That creates strong scarcity language, but it also creates a single future moment when the protocol stops providing any base miner reward at all. If transaction fees do not rise quickly enough to offset declining subsidy, security assumptions can become tighter precisely when the chain is older and more valuable.

Atho chooses a different compromise: four explicit bootstrap reductions followed by a permanent reward floor. The network still gets declining relative inflation, but it does not need a terminal-dust special case and it does not end in a subsidy value of zero. That choice trades away hard-cap branding in exchange for a simpler long-horizon miner-security story.

13.10. Wallet and User Display

Wallets and user-facing tools should display Atho with 8 decimal places:

- 1 ATHO = 100,000,000 atoms
- 0.00000001 ATHO = 1 atom
- 8 ATHO = 800,000,000 atoms
- 4 ATHO = 400,000,000 atoms
- 2 ATHO = 200,000,000 atoms
- 1 ATHO = 100,000,000 atoms
- 0.50 ATHO = 50,000,000 atoms

This display choice keeps consensus and user presentation aligned. The same integer atoms used by validation are the units wallets can convert back into familiar decimal amounts. Exchanges, explorers, and accounting systems do not need to infer or normalize a different public precision from the backend consensus model.

The authoritative fields for the current policy are the current reward, current bootstrap era or tail phase, tail-start height, blocks-per-day and blocks-per-year values, scheduled total issuance, circulating supply, burned supply, immature coinbase supply, annualized issuance, and the explicit emission-model label. Public reports

should not revive stale `max_supply_*` aliases or label scheduled issuance as circulating supply when those values are not actually equivalent.

Wallet fee controls should also present the distinction between the required floor and the optional extra tip. A sender should understand that:

- the minimum valid fee is consensus-enforced
- higher fee above that floor is discretionary
- miners are free to prefer higher-fee transactions during congestion
- extra TX-PoW does not replace fee priority

13.11. Final Policy Statement

"Atho uses a zero-emission genesis anchor at height `0`, begins ordinary miner subsidy at height `1`, pays `8`, `4`, `2`, and `1` ATHO across four `1,250,000`-block bootstrap eras, then pays a permanent `0.50 ATHO` tail reward from height `5,000,001` onward. Atho has no premine, no maximum supply limit, 8 decimal places, a required non-coinbase fee floor of `max(600 atoms, 1 atom per serialized byte)`, optional additional miner tip above that floor, and deterministic TX-PoW sender friction. The emission model is designed to provide predictable bootstrap issuance, a clear tail-activation milestone, and a persistent miner-security floor rather than a fee-only subsidy cliff."

14. Mempool Design

The mempool is the staging area for policy-valid transactions that have not yet been mined into a block. In Atho, the mempool should never become an alternate consensus system. Its job is narrower:

- reject malformed or consensus-incompatible transactions before they waste miner or relay resources
- track pending conflicts
- expose a validated pending set for block template construction
- provide useful status information to local tooling and API surfaces

The mempool should be stricter than “accept anything that might later work.” Loose pending-state logic creates relay noise, miner waste, and confusing wallet behavior.

The current node configuration gives the mempool explicit bounded limits: the default cap is `50,000` transactions and `64 MiB` of transaction vbytes, with operator overrides clamped into a bounded range. Mining views revalidate entries against current UTXO state instead of blindly trusting cached admission results.

14.1. Mempool Admission Flow

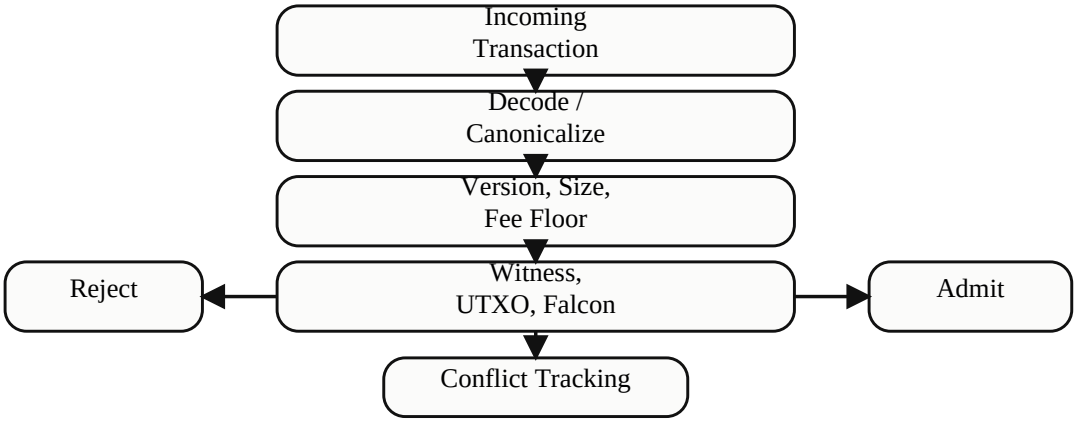


Figure 8. Mempool Admission Flow

An incoming transaction must pass decode, canonical structure, version rules, size policy, dust rules, TX-PoW checks, witness shape checks, UTXO availability checks, Falcon verification, and local conflict tracking before it is admitted.

15. Consensus Validation

Consensus validation is where Atho becomes a blockchain rather than just a messaging system. The network does not accept data because a miner produced it, a wallet broadcast it, or an API submitted it. The network accepts data because the local node can prove it is valid under canonical rules.

Transaction validation and block validation are related but distinct. Transaction validation checks canonical structure, ownership, spendability, optional tip accounting, TX-PoW, and witness correctness. Block validation adds header rules, proof-of-work, coinbase correctness, duplicate-spend detection, commitment verification, and atomic state transition requirements.

Table 10 lists representative invalid cases that must continue to fail identically across node, mempool, mining, and API entry points.

Table 10

Consensus-Critical Invalid Cases

Invalid Case	Rejection Surface	Why It Is Consensus-Critical
Wrong-network block or transaction	Header/network checks and address decode paths	Prevents cross-network acceptance and replay.
Legacy or noncanonical lock format	Output lock parsing and UTXO ownership validation	Prevents ambiguous or anyone-can-spend behavior.
Malformed witness, public key, or signature	Witness parser and Falcon verification	Prevents unauthorized spends and parser ambiguity.
Duplicate inputs or duplicate spends	Transaction and block contextual validation	Prevents explicit double spends.
Overpaid coinbase	Block monetary validation	Preserves deterministic issuance.
Wrong transaction or block version	Version checks against active ruleset	Prevents unactivated rule paths from entering validation.
Bad merkle or witness commitment root	Block commitment verification	Prevents tampering with the transaction set or witness set.
Oversized block or transaction	Size and weight limits	Preserves resource bounds and deterministic relay/validation rules.

15.1. Block Validation Pipeline

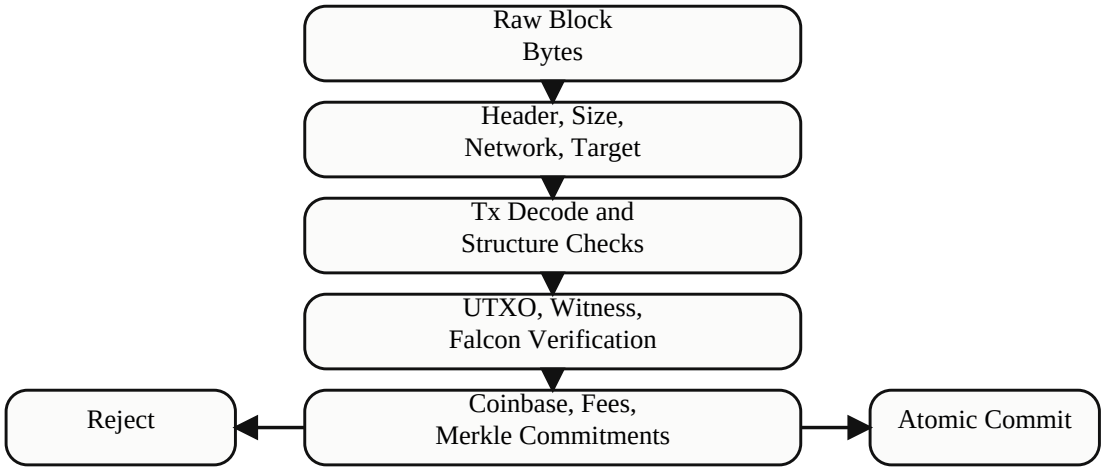


Figure 9. Block Validation Pipeline

The validation order should reject cheap failures first: malformed bytes, wrong network, bad header structure, oversized objects, coinbase violations, missing UTXOs, malformed witnesses, incorrect signatures, overpaid subsidy, and bad commitments. Expensive work should occur only after cheap structural rejection paths are exhausted.

15.2. Validation Pipeline and Parallel Work Distribution

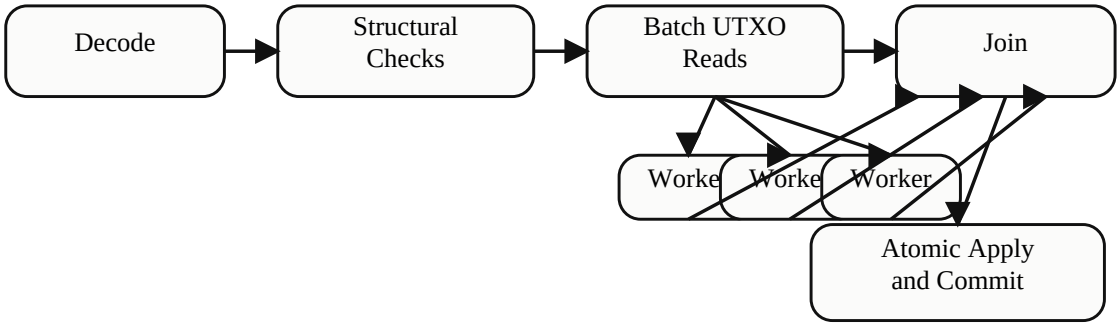


Figure 10. Validation Pipeline and Parallel Work Distribution

The current codebase already contains room for safe parallelism in independent signature verification and block-level work distribution. Parallel verification is valuable only if it remains deterministic and does not alter final state-application order. Structural checks and UTXO batch reads can prepare work in parallel, but final chainstate mutation must remain authoritative and ordered.

15.3. Authorization, Duplicate Identity, and Accounting Flow

The shared contextual transaction path is used by mempool admission and block validation. It resolves every input, checks duplicate references and checked amount sums, reconstructs every signer digest, verifies exact canonical Falcon-512 encodings, and requires complete signer coverage before returning a fee. No alternate minimum-fee path is permitted to skip authorization.

```
canonical transaction bytes
          |
          v
structure / version / size / dust / TX-PoW
          |
          v
resolve every outpoint -> reject missing, immature, or duplicated input
          |
          v
checked sum(inputs) and checked sum(outputs)
          |
          v
parse canonical grouped witness
          |
          v
exact Falcon lengths + canonical backend encoding
          |
          v
rebuild digest for each covered input group and verify signature
          |
          v
all inputs covered exactly once? -> derive fee -> enforce fee floor
```

At block scope, the transaction identifier set is checked before UTXO mutation. The coinbase identifier is made height-specific through the exact `lock_time` commitment, and live outpoint insertion is preflighted before writes. Disconnect uses recorded undo state, preserving the invariant that connect followed by disconnect restores the prior usable-output set.

Merkle and witness-root construction also reports mutation metadata without changing the root bytes. Equal real siblings at any tree level fail validation; the synthetic duplicate used only to pad an ordinary odd layer does not. Sync requires matching roots, unique txids, and non-mutated trees before an invalid body may mark its header hash invalid.

```
candidate block
  -> exactly one coinbase at index 0
  -> coinbase lock_time == exact block height
  -> all txids unique and all spent outpoints unique
  -> validate every spend and checked amount transition
  -> coinbase total == scheduled subsidy + validated fees
  -> atomically apply UTXO batch and persist undo
```

15.4. Bitcoin-Inspired Hardening, Atho-Native Rules

The historical Bitcoin proposals used during this review are design references, not wire-format claims. Atho keeps its own SHA3, Falcon, witness, transaction, header, and activation structures.

Reference principle	Atho implementation
BIP 30 duplicate transaction safety	Duplicate block txids fail, live outpoints cannot be replaced, and archive recovery is keyed by block plus transaction index.
BIP 34 coinbase uniqueness	Canonical coinbase `lock_time` equals the exact `u32` block height.
BIP 66 canonical signatures	Falcon public keys and signatures use exact fixed lengths plus the backend canonical encoding gate.
BIP 147 malleability closure	Atho has no Bitcoin script dummy value; canonical grouped witnesses require complete input coverage and are committed through the header witness root.
BIP 9 upgrade safety	A validated fixed-height schedule is used; no miner-signaled state machine is active.
BIP 130 direct header relay	Bounded header responses and compact tip announcements carry headers; branch-context validation precedes body download. Atho does not claim Bitcoin `sendheaders` wire negotiation.
BIP 144 witness services	Full-block and witness services are mandatory, and stripped bodies cannot match the witness commitment.
BIP 152 compact relay	Header-keyed full-identity short IDs, collision-safe lookup, exact missing indexes, and full-block fallback.

16. Network Layer

Atho exposes separate network identities for `mainnet`, `testnet`, `regnet`, and `prunetest`. Each has its own consensus ID, genesis anchor, visible address prefix, port set, and P2P magic values. This matters because network isolation is not a convenience feature. It prevents one network's artifacts from being accepted as another network's truth.

The founder-hash metadata is fixed across all supported network anchors. The active founder hashes are `3582d92eff98ba5e575ae721ced2b32bf7bea4628971d339dc189fde10716a40b91a67f8eb300894eaebb921754cdd86` for SHA3-384 and `c93086ce58c4a64ef15672b00836eca94d14ea3810a584360fb11e11f065d5949aa1307f9315bce31316c820e6f58f0dace98f48111e6538a3d7a231e58e51bf` for SHA3-512. The current mainnet, testnet, regnet, and prunetest genesis anchors preserve those values.

Network	Genesis Hash	Coinbase Txid / Merkle Root	Nonce
mainnet	`0000f1fc7e66238f4068201c3c25b80fa9e2ce8a6df744ee4fd1e0984c7fa9ebd546028acd3d81d0c4a7e8c0d17a9b1e`	`508c56d90840562f88a861391bb1f44838652af65682e78ffd78fca64d5ba06375a337cfbb4df9700fc6e4e22407263c`	54,366
testnet	`00002eb00a5fc1d177b03a8b77348c5c1faef451fa9367a337863df7af37f61734ed202e321353fd80a319843bcfef6b`	`7b463dd423fa97095d27f21d42ce5169117814373f5b9eba89b84b80f8d4ab337cbf04aafa329326291509dc378de8d9`	140,842
regnet	`00000bc9bd6fff756519b8e62f89a4baef483e7b297009bd9a574692c9e58ba442dc9418628fc1039d4856e086d5179b`	`7b463dd423fa97095d27f21d42ce5169117814373f5b9eba89b84b80f8d4ab337cbf04aafa329326291509dc378de8d9`	1,442
prunetest	`00002f76f518da66ae58f36498138606fbc48f8a35b0e931ee9af6696651260bdf91f68c310a7cb50779800b787fe3c5`	`7b463dd423fa97095d27f21d42ce5169117814373f5b9eba89b84b80f8d4ab337cbf04aafa329326291509dc378de8d9`	40,204

The P2P layer is responsible for peer connections, version exchange, inventory relay, headers-first synchronization, block retrieval, and invalid-data rejection. It is not responsible for deciding consensus truth independently of the validation engine.

16.1. Node Sync and Block Propagation Flow

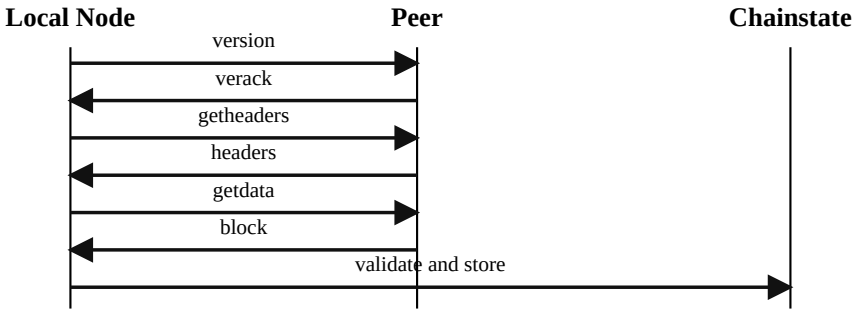


Figure 11. Node Sync and Block Propagation Flow

Headers-first synchronization allows a node to learn candidate chain shape before fetching full block payloads. After header exchange, the node requests blocks, validates them locally, persists accepted state, and only then advertises the new tip. Propagation should never outrun validation.

16.2. Canonical Tip Extension and Competing Branch Handling

The current sync path distinguishes between one exact next-block context and every other branch candidate. That distinction matters because the adaptive raw-byte limit is only well-defined for the block that directly extends the local canonical tip.

When a received block points to the current canonical tip, the node can apply the active adaptive raw-byte limit immediately as an early envelope check before deeper contextual work. When a received block belongs to another branch, the node still enforces hard network, decode, raw-byte, vbyte, and weight caps right away, but it buffers that block for later ancestry and accumulated-work comparison instead of judging the branch only by the local tip limit. This keeps fast rejection for malformed or impossible traffic without turning a local next-block budget into a universal branch-rejection rule.

```
Canonical Tip Extension and Competing Branch Flow

  peer announces headers / block
              |
              v
  decode, network, and hard-cap checks pass?
      |
      +-- no  -> reject immediately
      |
      +-- yes
              |
              v
  previous hash == local canonical tip?
      |
      +-- yes
      |       |
      |       v
      |   apply current active adaptive raw-byte limit
      |       |
      |       +-- exceeds limit -> reject tip extension
      |       |
      |       +-- within limit -> full validation -> new canonical tip
      |
      +-- no
              |
```

```
            v
  treat as competing branch candidate
            |
            v
  buffer block and re-evaluate later with ancestry,
  chainstate context, and accumulated work
```

16.3. Contextual Headers and Compact Block Recovery

A sync peer must advertise both full-block and full-witness service capability. Received header batches are bounded and staged atomically. Before any staged header becomes a download candidate, the node checks continuity, exact height, active version, network and founder anchors, timestamp drift, median time past, the exact next target derived from branch history, and SHA3-384 proof-of-work.

Timestamp validation deliberately separates deterministic consensus replay from live-network policy. Historical chainstate validation uses parent timestamps, median time past, target history, and UTXO state; it does not compare old blocks with the validator's current wall clock. Live P2P, synchronization, and local submission paths reject or defer blocks more than two hours ahead of local time. This future-time rule is an ingress policy boundary, while median-time ordering remains the reproducible historical rule. Operators must keep clocks reasonably synchronized and preserve the policy across every block entry path.

```
version/verack -> require full-block + witness services
       |
       v
bounded header batch -> validate every header in branch context
       |
       +-- any failure -> reject the whole staged batch
       |
       v
cache validated headers -> schedule bodies
```

Compact relay uses a key derived from the announced canonical header and short identifiers derived from full witness commitment identities. This means witness or TX-PoW variants do not alias as the same relay object. A duplicated short ID or mempool collision becomes an explicit missing transaction. Recovery accepts only the exact ordered indexes requested and only from the originating compact peer; an incomplete, malformed, cross-peer, or still ambiguous response is rejected or triggers full-block download.

```
compact block
  -> validate header and compact structure
  -> require coinbase prefilled at index 0
  -> map unique header-keyed short IDs from mempool
  -> request every missing or collided index
  -> require exact ordered blocktxn response
  -> recompute merkle root and witness root
  -> same authoritative full-block validation

any reconstruction ambiguity or commitment mismatch -> request full block
```

A malformed or commitment-ambiguous body cannot poison an independently valid header hash. If roots do not match, txids repeat, or equal real tree siblings are detected, the peer is penalized, the hash is retried from another peer, and the work is requeued without adding that hash to the permanent invalid set. This prevents a peer from attaching corrupt transaction data to a known header and suppressing later download of the genuine block.

16.4. Runtime Resource and Topology Policy

Network resource controls are local transport policy rather than consensus. Each connection has bounded byte, message, transaction, header, and expensive-request accounting. Outbound delivery is bounded and prioritizes handshake, keepalive, headers, and block recovery over transaction relay. Decoded messages enter a bounded dispatcher before serialized node-state validation, preserving deterministic chainstate mutation while limiting queued work.

Outbound selection persists recent healthy anchors and periodically opens one temporary feeler connection to a non-anchor candidate. Candidate ordering seeks IPv4, IPv6, DNS-family, subnet, and address-source diversity when those options exist. Peer quality separately records useful blocks, headers, transactions, invalid responses, stalls, resource violations, queue drops, and validation latency. A peer is not rewarded merely for producing high traffic volume.

17. Storage Layer

The current storage model is hybrid. Canonical raw block bytes are archived in flat block data files, while LMDB stores indexed metadata, UTXO state, chain metadata, transaction archives, and related lookup structures. This mirrors a practical separation of concerns:

- flat files are efficient for canonical historical payload storage
- LMDB is efficient for current indexed state and metadata lookups

Storage remains consensus-adjacent because partial or unsafe commits would create false local chain truth. Accepted blocks must therefore update state durably and coherently.

17.1. Hybrid Storage Commit Model

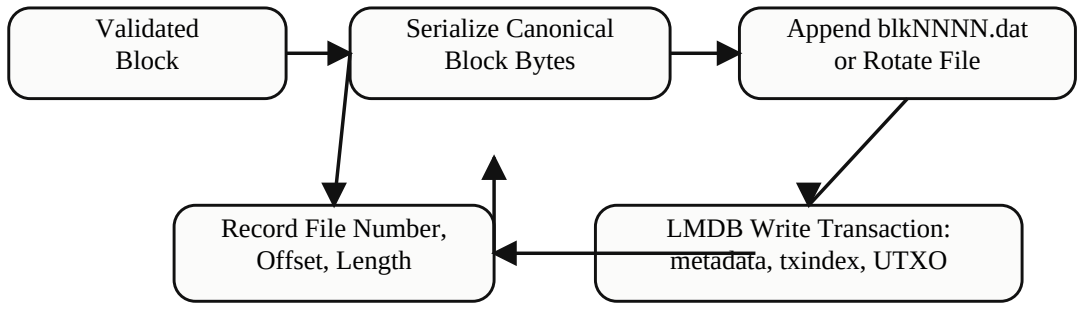


Figure 12. Hybrid Storage Commit Model

After a block is validated, the node serializes canonical block bytes, appends them to the active flat block archive or rotates to the next file, records file offsets, and then performs an LMDB-backed write transaction for metadata, indexes, and UTXO changes. The important design property is not the file layout by itself; it is that accepted local truth remains reconstructable and queryable without redefining consensus.

18. Wallet Architecture

Atho wallets derive keys, build addresses, select UTXOs, estimate optional tips, create change, sign transaction witnesses, and track balances relative to node-reported state. Wallet code should be conservative: it should never create transactions that consensus rejects under normal conditions when all necessary local data is available.

The live repository also separates wallet responsibilities from node responsibilities. The wallet owns keys and local intent. The node owns validation, relay, mempool policy, and block acceptance.

18.1. Wallet-to-Node Interaction

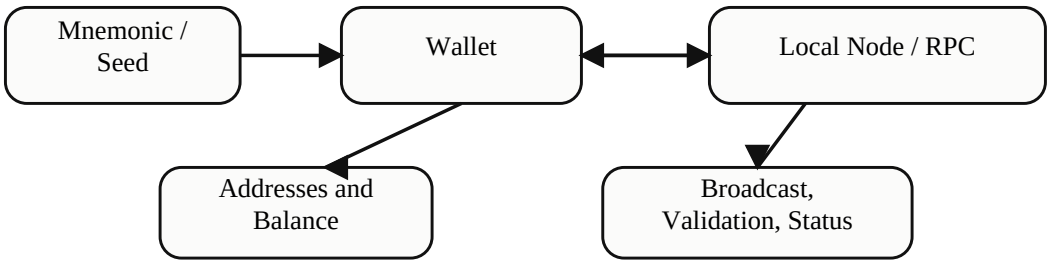


Figure 13. Wallet-to-Node Interaction

The normal wallet flow begins with mnemonic or seed material, derives local keys and receive/change addresses, builds a candidate transaction, signs it with Falcon-512, submits it to the local node or API surface, and then listens for status updates, balance changes, and confirmation depth through validated node state.

19. Wallet-Level Privacy and Linkability Reduction in the Alpha Implementation

Atho's alpha privacy posture is deliberately wallet-level and transparent-chain aware. The current repository does not implement hidden amounts, hidden receivers, ring signatures, stealth-address scanning, shielded notes, confidential transactions, transaction mixing, Tor routing, Dandelion relay, or a consensus privacy pool. Atho is a public UTXO network: transaction inputs, transaction outputs, output amounts, txids, block placement, coinbase rewards, and the movement of spendable outputs remain visible to a normal full node and to any public explorer built on validated chain data. That fact is not a weakness to hide in the white paper. It is the boundary that makes the implemented privacy features understandable and honest.

The alpha implementation improves user privacy in a narrower but still meaningful way: it reduces avoidable address reuse, makes fresh receive and change addresses cheap, separates receive and change derivation paths, lets the wallet split change into several HD change outputs, supports multiple recipient outputs in one transaction, accepts lower-level requests built from selected UTXOs, tracks unused and used wallet addresses, keeps wallet signing local, encrypts wallet datafiles by default in normal UI flows, and can rotate mining rewards to fresh receive addresses. These tools do not make a transparent transaction anonymous. They do make the wallet less likely to expose a single static address as the user's long-term identity, and they give advanced users practical controls for managing UTXO shape before stronger protocol-level privacy exists.

The most important distinction is between privacy and unlinkability. Privacy is the broad goal of limiting what other people can learn. Linkability is a narrower chain-analysis question: can an observer infer that two outputs, two addresses, two receives, or two spends belong to the same wallet or actor? Atho alpha cannot prevent every

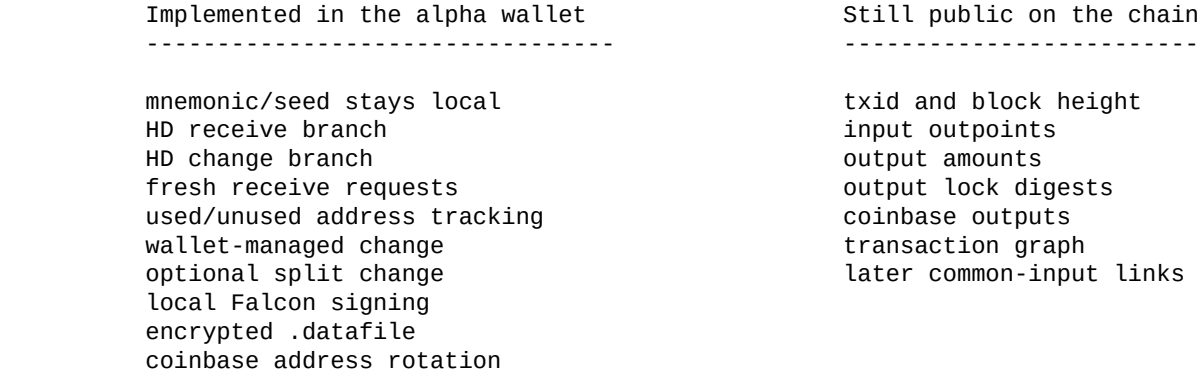
link. It can prevent unnecessary links that come from using one address forever, sending all change back to a reused public destination, mining every reward to a single payout address, or immediately combining unrelated UTXOs without user awareness. In this paper, "privacy improvement" means linkability reduction and local secret protection, not mathematical anonymity.

The current wallet privacy design has four layers:

- address hygiene: deterministic HD receive and change branches, a large prefilled keypool, address labels, visible receive indexes, unused/used address pool filters, and payment-request address generation
- transaction-shape controls: multiple recipients, custom or wallet-managed change, split change across multiple HD change outputs, tip-in-total handling, dust avoidance, and transaction-size checks
- spend safety: local UTXO scanning, mempool-reserved-input exclusion, wallet confirmation policy, coinbase maturity awareness, grouped Falcon signing, and transaction proof-of-work stamping
- local operational protection: mnemonic and seed redaction in debug output, zeroizing seed material, password-backed AES-256-GCM wallet files, owner-only file permissions on Unix, local RPC defaults, and opt-in administrative/wallet API surfaces

Figure 14 shows the current high-level privacy boundary. The left side contains implemented alpha wallet behavior. The right side contains facts that remain public because Atho is still a transparent UTXO ledger.

Figure 14. Alpha Wallet Privacy Boundary



19.1. Threat Model for the Current Wallet Privacy Tools

The privacy tools in Atho alpha are intended to help against ordinary passive graph observation and local secret exposure. A passive graph observer watches blocks, transactions, UTXOs, addresses, and amounts. That observer can see when two inputs are spent together. It can see when a transaction creates one payment-sized output and one change-looking output. It can see every coinbase payout and every later spend of that payout. It can sort addresses by reuse and identify addresses that receive many unrelated payments. The alpha wallet cannot erase those facts, but it can reduce the number of obvious, self-inflicted signals.

A local secret exposure adversary is different. That adversary tries to steal the wallet seed, mnemonic phrase, password, datafile contents, or locally cached private material. The repository addresses that risk through encrypted wallet persistence, zeroizing secret wrappers, debug redaction, owner-only wallet file permissions on Unix, and UI/runtime defaults that treat encryption as the normal wallet mode. These controls protect wallet material at rest and during routine operation, but they cannot protect a seed once the user exports it insecurely, types it into another program, stores screenshots, or runs the wallet on a compromised operating system.

A network observer is different again. If a transaction is first announced from the user's node, a peer or network monitor may infer that the user originated it. The current codebase does not implement a special transaction-origin

privacy layer. There is no committed anonymity network, stem/fluff relay, random broadcast delay, or built-in Tor-only mode in the audited alpha code. Therefore this section does not claim network-layer privacy. The safest wording is that the alpha wallet improves address and UTXO hygiene while still relying on normal node relay behavior.

A remote API operator is also relevant. The desktop wallet is designed around a local node, and local scanning keeps wallet decisions close to the user. However, if a user connects wallet tooling to a remote RPC/API endpoint, that remote service can learn more about the wallet's interests. In particular, wallet activity lookups submit wallet address data to the connected service. Local-node use avoids that third-party disclosure. This is a user-operation boundary, not a consensus boundary.

Table-style summary of the current alpha privacy posture:

Privacy Area	Implemented Alpha Behavior	What It Does Not Hide
Address reuse	Fresh HD receive and change addresses are easy to generate and track.	Reusing a digest still links all payments to that digest.
Change handling	Change can go to wallet change addresses, a custom address, or multiple split change outputs.	Observers still see the transaction that created the change.
UTXO shape	The builder can spend selected UTXOs, and the alpha GUI exposes manual coin control with labels, freeze flags, smart selectors, and merge warnings.	Manual selection does not hide inputs, amounts, or transaction graph history.
Mining rewards	The desktop wallet can rotate coinbase rewards to fresh receive addresses after accepted blocks.	Spending those rewards together later can re-link them.
Local secrets	Mnemonics, seeds, and datafiles receive local protections.	Compromised hosts, insecure exports, and plaintext wallets remain risks.
Network origin	Local node operation avoids outsourcing wallet state to a third-party wallet service.	Transaction broadcast origin is not hidden by a special relay protocol.
Amount privacy	None at consensus level. Amounts remain public integer atoms.	All output values are visible.

19.2. HD Wallet Structure, Receive Paths, and Change Paths

Atho alpha wallets are deterministic HD wallets in the practical sense used by the current codebase: one seed can reproduce a sequence of receive and change addresses. The implementation is intentionally simple. A wallet is created either from a mnemonic phrase plus optional mnemonic passphrase, or from a raw `WalletSeed`. Mnemonic-derived wallets compute a 64-byte root seed with PBKDF2-HMAC-SHA512 under the `atho-mnemonic-v1` scheme and `600,000` iterations. The wallet then hashes that root seed with SHA3-256 to obtain the 32-byte wallet seed used by the HD wallet object. The default mnemonic length is 24 words, while the mnemonic module supports 12-, 24-, and 48-word phrase lengths with checksum validation.

The HD path structure currently contains an account number, an address kind, and an index. The active account value is `0`. The address kind is either `Receive` or `Change`. Receive and change counters advance independently. That matters for privacy because change addresses do not have to reuse the same visible addresses that users hand to payers. A simple wallet that sends change back to the original receiving address creates a long-lived public cluster. Atho's alpha wallet instead has a dedicated change branch and a transaction builder that can direct change to one or more change-branch outputs.

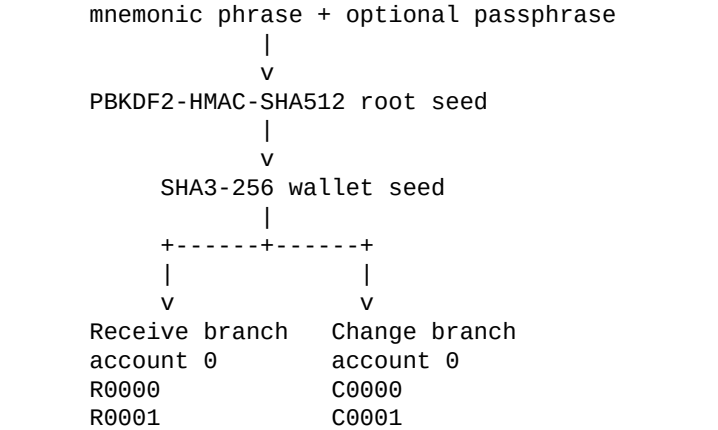
The key derivation code also mixes the active network into key generation. The Falcon keypair seed is built from the wallet seed, the network domain tag, the account, the receive/change role byte, and the index, then hashed through SHA3-384 before deterministic Falcon key generation. This prevents silent key reuse across mainnet, testnet, regnet, and prunetest, and it prevents receive and change roles from accidentally sharing the same key stream. The visible address prefix also encodes the network, but the privacy value starts before the display string: the key derivation itself is network- and role-separated.

Each derived address has a Falcon public key, a 32-byte payment digest derived from that public key with an address role domain and network domain, a visible Base56 address, a network prefix, a checksum, and an internal hashed-public-key string. Consensus outputs lock to the 32-byte payment digest. The public key is revealed later in the transaction witness when the output is spent. This gives unspent outputs a compact digest lock instead of publishing a full 897-byte Falcon public key at receive time. It does not hide that the same digest was reused, but it does keep the public key itself out of the UTXO until spend.

The wallet pre-fills a keypool of receive and change paths. The current target is `1,024` receive paths and `1,024` change paths. On checkout, the wallet takes the next path from the appropriate queue, derives the address, records it in the address book with an optional label, refills the keypool back toward target, and updates a wallet snapshot count. This makes fresh address generation cheap at the user interface layer. A wallet can create a new receive address for a payment request without pausing to derive a long chain of addresses synchronously.

Figure 15 summarizes the alpha HD structure.

Figure 15. HD Address Separation in Atho Alpha



```
R0002            C0002
     |             |
     v             v
Base56 receive   Base56 change
address          address
```

The privacy benefit is address availability. Users do not need to decide whether a new address is "worth it." The wallet architecture makes a new address normal. This matters because most transparent-chain privacy failures begin with address reuse: one donation address posted publicly, one mining address used forever, one merchant address reused by every invoice, or one change address reused over many sends. HD generation does not make transactions private by itself. It makes better behavior cheap enough to become the default.

19.3. Receive Address Rotation, Payment Requests, and Address Pool Visibility

The receive page exposes fresh-address behavior directly. The user can create a new receiving address, attach an optional local label, optional requested atom amount, and optional local message, and display a QR code for the resulting request. The request history stores the address and metadata locally for user recall. The current receiving address is displayed with a wallet index such as `R0000`, and the address pool tab shows receive and relevant change entries with path labels, role labels, and usage status.

This is important because privacy is partly an interface problem. A wallet can have good derivation code but still encourage users to copy the same address repeatedly. Atho's alpha interface gives users an obvious path for making a fresh receiving address for each payer or each invoice. It also tells the user whether a loaded address is unused or used. The address pool filter defaults to unused addresses and can also show used or all loaded addresses. This helps users avoid accidentally handing out an address that already has public history.

The address pool does not display the entire infinite HD space. It displays addresses loaded into the wallet discovery window. The default restore gap limit in the wallet library is `1,000`. The desktop client begins with quick scan steps and can expand the active scan window through preset steps up to `1,000`, while settings allow a higher recovery window up to the built-in maximum used by the UI path. This approach balances startup cost against restore safety. From a privacy standpoint, the important fact is that both receive and change addresses can be discovered again from the seed and scan window, so rotating addresses does not mean the user must back up every new address independently. The seed and counters are enough when recovery scanning is configured to cover the used range.

The wallet's local scan path requests UTXO data from the connected node and matches output locking scripts to locally derived payment digests. It then builds receive-address rows by counting UTXOs and atom totals for each digest. This lets the wallet show whether a receive address has on-chain use without embedding a central address-tracking service into the wallet model. When the node is local, that matching remains local. When a user chooses a remote backend, the remote backend already has visibility into whatever request surface the wallet uses, so remote-wallet operation should not be described as equivalent to local-node privacy.

The ability to create new payment requests also supports a common operational pattern: one address per invoice, one address per mining session, one address per counterparty, or one address per public campaign period. In a public UTXO system, this does not prevent a later spend from linking those outputs if the wallet combines them. But it does prevent every payer from seeing every other payer by simply pasting the original address into an explorer. That is a practical privacy improvement for merchants, miners, donors, testers, and ordinary users.

The alpha receive flow also exports QR codes. A QR code is just another representation of the visible Base56 address. It does not add cryptographic privacy. It does reduce copy/paste mistakes and makes one-time addresses easier to use. Users should still treat a QR code as public if they share it, print it, stream it, or save it in a public folder.

19.4. Address Encoding, Public-Key Exposure, and Wrong-Network Protection

Atho addresses encode a 32-byte payment digest into a Base56 string with a visible network prefix and a fixed checksum suffix. Mainnet, testnet, regnet, and prunetest have distinct visible prefixes. Address decoding rejects non-ASCII input, invalid prefix characters, invalid alphabets, short strings, and checksum mismatches. Send, custom change, and mining reward paths all validate that a supplied address belongs to the active network before accepting it.

Wrong-network rejection is not usually called a privacy feature, but it protects users from a real leak: accidentally publishing or using an address in the wrong operational context. A testnet address pasted into mainnet tooling should fail before transaction construction. A regnet mining address should not silently become a mainnet reward target. The network prefix and checksum also make address handling less ambiguous for external tools, exchange integrations, and scripts. Privacy is often lost through operational mistakes, not only through cryptographic weakness.

The internal lock is the digest, not the visible string. This separation helps the code keep consensus state compact and unambiguous. It also means a visible address can be treated as a user-interface encoding instead of a script language. The current consensus locking script for spendable outputs is canonical only when it is exactly the 32-byte payment digest. Non-canonical lock formats are rejected by wallet construction and validation paths. A narrow lock format reduces the chance that alternate encodings or legacy forms create hidden address reuse or parser ambiguity.

Because the payment digest is derived from the Falcon public key under an address role domain and the network domain, the output does not reveal the full public key until spend. Once a user spends that output, the witness includes the public key and signature needed for validation. Observers can then verify that the key matches the previous digest. This is normal transparent-chain behavior: the output is compact before spend, and the authorization material appears at spend. It does not stop observers from linking all outputs sent to the same digest. The right user behavior is still to avoid digest reuse.

19.5. Transaction Builder Privacy Controls: Recipients, Change, and Output Count

The alpha send flow exposes several transaction-shape controls that matter for privacy and UTXO management. The primary recipient is required. Extra recipient rows can be added until the standard output cap is reached. Each row becomes another recipient output. The wallet also supports a mode that includes the total fee in the typed amount, so a user can decide whether the displayed amount is the full wallet debit or the recipient amount before fees. The fee choice is reviewed in a confirmation dialog, and the final send confirmation shows total sent, total fee, wallet debit, size, output count, and change behavior.

Multiple-recipient support is useful for ordinary batching and for controlled self-organization. A user can pay several recipients in one transaction, or can include outputs to addresses the user controls. That is not the same as mixing. Every output in that transaction is visible in the same public object. If the transaction pays one external recipient and several self-controlled outputs, observers may not know which output is which with certainty, but they can still analyze amounts, future spends, and change patterns. The feature is best understood as transaction construction flexibility, not anonymity.

Change handling is more directly privacy-relevant. When a UTXO has more value than the amount being paid plus the selected total fee, the remainder needs to go somewhere. If it returns to the same public address that received the funds, the wallet creates an obvious reuse trail. Atho's wallet-managed change path instead checks out change addresses from the HD change branch when change is needed. The send page exposes change modes: wallet default change, new unused wallet address, and custom change address. In the current submit path, wallet-managed modes reserve wallet change addresses at send time when change is actually created. Custom change lets the user paste a Base56 address, but the code rejects wrong-network custom change before building

the transaction.

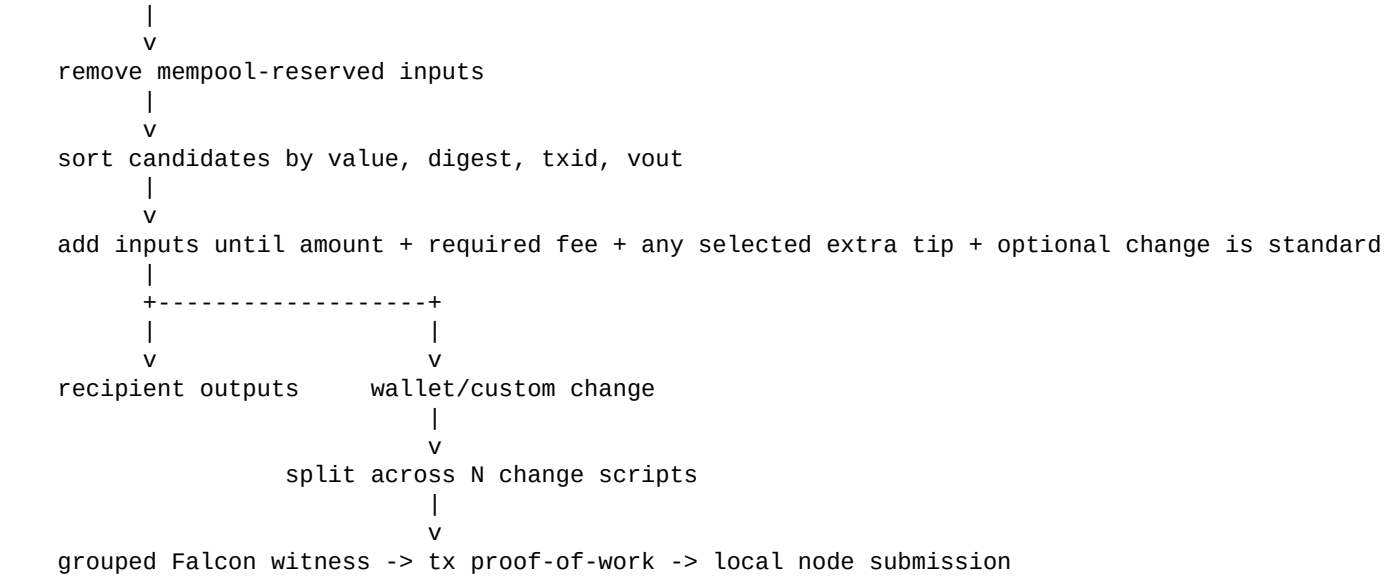
The wallet also lets the user choose a number of change outputs, bounded by the maximum standard output count after recipient outputs are counted. If change exists, the builder can split it across the selected change locking scripts. The split is even: base amount equals total change divided by output count, and remainder atoms are distributed one by one to the earliest outputs. The builder rejects a split if any resulting change output would be below the dust floor. This prevents a privacy-motivated split from creating nonstandard dust outputs that the node or mempool would reject.

Split change gives users a way to avoid creating one large change output that later dominates wallet behavior. For example, a user who spends from a large UTXO can split the leftover value into several change outputs. Later sends can draw from smaller pieces instead of repeatedly dragging the entire large output into transactions. This can reduce future over-linking caused by one oversized change output being reused as the obvious funding source for many payments. However, split change also creates several outputs in the same transaction, and those outputs are initially linkable by common origin. The privacy benefit appears only if later wallet behavior uses those outputs carefully and does not recombine them immediately.

The lower-level wallet transaction builder accepts `selected_utxos`, `recipient_outputs`, `change_address`, `change_locking_script`, and `change_locking_scripts`. The alpha GUI now exposes that builder surface through a compact coin-control panel in the Send workflow. Automatic planning remains available, but manual mode routes the user-selected outpoints into the strict selected-input planner so the builder receives the exact UTXO set the user chose.

Figure 16 summarizes the implemented send path.

Figure 16. Alpha Send Builder and Change Split Flow



19.6. Automatic and Manual UTXO Selection

The desktop wallet's automatic UTXO selection is deterministic and conservative. It starts from spendable wallet inputs, excludes mempool-reserved inputs, applies wallet confirmation policy, sorts candidates by value descending, then by payment digest, txid, and output index, and incrementally adds candidates until it can satisfy the recipient amount, selected fee target, dust floor, standard output count, and standard transaction size. It then prefers candidates with lower estimated fee, fewer inputs, fewer signer groups, fewer outputs, and lower total input value when other factors tie.

This policy has privacy tradeoffs. Fewer inputs usually means fewer common-input links, because spending multiple inputs together is one of the strongest public signals that the same wallet controls them. Fewer signer groups can also reduce witness size and complexity. Lower total input value can reduce unnecessary exposure of unrelated balance. At the same time, value-descending selection can spend large outputs first, and a deterministic policy can have recognizable patterns. The alpha wallet chooses reliability, fee control, and simple behavior over a randomized coin-selection privacy strategy.

The wallet avoids spending outputs that are already reserved by the mempool. It asks the node for mempool-spent inputs and removes them from wallet spend candidates. This is primarily double-spend safety, but it also prevents the wallet from accidentally building confusing follow-up transactions from outputs that are already in flight. The wallet also waits for its confirmation policy before marking outputs available. Consensus says normal transactions become spendable after one confirmation, while the default wallet filter is three confirmations. Users can choose other thresholds in settings. From a privacy standpoint, waiting for confirmation reduces accidental churn around unfinalized history and keeps the wallet from constructing spends against outputs that may disappear during sync or mempool changes.

Coinbase outputs have separate maturity rules. Consensus requires 100 blocks before a coinbase output is spendable. This is primarily a consensus stability rule, but it shapes miner privacy because a miner cannot immediately churn every new reward through fresh addresses in the next block. Coinbase rotation therefore creates separate reward outputs first, while later spending behavior determines whether those outputs remain separate or become linked.

Manual coin control adds user-selected outpoints to that existing automatic foundation. The Send page lists wallet-owned UTXOs with amount, HD path, label, output type, confirmations, spend status, privacy status, and freeze controls. Frozen outputs are excluded from automatic candidates. When manual mode is enabled, the selected UTXOs are planned as a strict set instead of letting the wallet add unrelated inputs silently. If that set cannot cover the recipients and selected fee target, the wallet refuses the send and asks the user to choose different coins.

19.7. Grouped Falcon Witnesses and Their Privacy Consequences

Atho transactions use Falcon-512 witnesses. The witness model supports one primary signer group plus additional signer groups. Each signer group has a signature, public key, and input references. The wallet groups selected inputs by their locking script, signs one digest for the input indexes in that group, and places additional signer groups into the canonical witness when a transaction spends outputs from multiple wallet addresses. This keeps the spend authorization precise: a signature covers the transaction base data and the indexes it authorizes under the active network and genesis context.

Grouped signing is not a privacy system. It does not hide which inputs were used. It does not hide how many address groups appear in the transaction. It does reduce witness overhead compared with signing every input separately when several inputs come from the same address digest. It also makes multi-address spends possible while keeping verification explicit. A node parses the witness, checks Falcon key and signature lengths, rebuilds the transaction signing digest for the covered input indexes, derives the payment digest from the public key, and verifies that the public key matches the referenced output lock.

The privacy consequence is subtle. A transaction with inputs from several address groups still publicly links those inputs through common spend. A transaction with several inputs from one address group reveals that those UTXOs share the same lock digest already. Grouped signing is therefore mainly a correctness and efficiency design, not a graph-hiding design. The wallet privacy benefit comes earlier, from not reusing addresses and from avoiding unnecessary input combinations where possible.

Signature prehashing includes the signature domain, network consensus ID, genesis hash, transaction base signing digest, and covered input indexes. This network binding prevents signatures from being reused across Atho networks. It also prevents a wallet or attacker from claiming a signature was intended for a different chain context. Again, this is not anonymity, but it is part of privacy-safe operation: signatures should not drift across environments in a way that exposes users to replay, confusion, or cross-network address handling mistakes.

19.8. Sending to Yourself, UTXO Splitting, and What It Can Actually Achieve

The send page includes a button that can fill the recipient field with the current receiving address. A user can also create a fresh receive request first, then send to that unused address. Together with extra recipient rows and split change, this gives the alpha wallet practical self-send and UTXO-splitting workflows. A user might consolidate small UTXOs, split one large UTXO into several outputs, move funds away from a reused receive address, or prepare a set of smaller wallet-controlled outputs for future payments.

These workflows improve privacy only when used with realistic expectations. A self-send from address A to fresh address B is visible as a transaction from an input locked to A's digest to an output locked to B's digest. Observers cannot prove intent from the chain alone, but they can see the direct graph edge. If the amount is distinctive, if the transaction has no external-looking recipient, or if B is spent soon after in a related pattern, the link may remain obvious. A self-send is not a mixer.

UTXO splitting has similar limits. Splitting one 100 ATHO output into ten 10 ATHO outputs can make future spends less likely to reveal a 100 ATHO balance every time. It can also create outputs with standard sizes that are less revealing than one unusual remainder. But all ten outputs are born in the same transaction, and an observer can track them as siblings. If the wallet later spends several siblings together, the link becomes stronger. Splitting is a tool for shaping future spends, not a way to erase history.

The alpha implementation helps keep self-send and split workflows standard. It enforces dust floors, maximum standard outputs, maximum transaction raw size, maximum vsize, correct optional-tip accounting, and transaction proof-of-work. It can split change evenly without creating atoms. It rejects missing change addresses when change is required. It rejects custom change that belongs to the wrong network. It solves transaction PoW after signing so the transaction is acceptable under the anti-spam policy. These checks matter because a privacy workflow that builds invalid transactions is not useful.

Best practice for alpha users is therefore conservative: use fresh receive addresses for unrelated receives, use wallet change instead of reusing receive addresses, avoid merging unrelated funds unless necessary, avoid immediate recombination after splitting, and remember that public amounts remain public. The wallet gives building blocks. It does not give a cryptographic privacy pool.

19.9. Coinbase Address Rotation for Miners

Mining rewards are some of the easiest outputs to cluster on a transparent chain. If every block mined by one operator pays the same visible address, all of those rewards are trivially grouped before any spending occurs. Atho alpha addresses this with two mechanisms. First, the node requires a valid configured mining reward address for active block template construction, and it rejects a reward address that belongs to another network. Second, the desktop wallet can rotate coinbase rewards to fresh receive addresses when the user enables the "Rotate coinbase to a fresh receive address" setting.

The node-side mining path builds a candidate block from the current tip and mempool. It computes the fixed consensus subsidy, gathers valid optional tips, decodes the configured reward address, verifies that the address network matches the node network, and uses the decoded payment digest as the coinbase locking script. Mainnet and testnet defaults intentionally do not provide a silent built-in reward address. Regnet and prunetest can use deterministic development reward addresses, but public-network mining requires the operator to configure an address. This prevents accidental mining to a wrong or hidden default on public networks.

The desktop wallet adds rotation around that node rule. When a mining job completes and the solved block is accepted, the UI checks the rotate-coinbase preference. If rotation is enabled, it checks out a new receive address from the wallet, appends it to the address views, marks it as the current receive address, updates the wallet snapshot, and persists the wallet state. Before future mining templates are requested, the wallet applies the current mining reward target through configuration and runtime RPC so the node builds the next candidate block to the current address.

Figure 17 shows the coinbase rotation loop.

Figure 17. Coinbase Rotation Loop

```
wallet current receive address
      |
      v
set miningrewardaddress / runtime target
      |
      v
node builds candidate block coinbase
      |
      v
miner solves and submits block
      |
      v
block accepted?
      |
      +-- no  -> keep current target
      |
      +-- yes -> checkout next receive address
                 persist wallet
                 use it for next template
```

The privacy benefit is straightforward: rewards do not all land on the same digest. The limitation is equally important: if the miner later spends many rotated rewards in one transaction, common-input analysis can cluster them again. Rotation improves the receive side of mining. Spend discipline still matters. The settings page warns that rotation is off by default and that rotating every mining reward can fragment spendable balance. That warning is honest. More separate rewards can improve receive-side unlinkability, but they can also create many UTXOs that later cost more to spend and may be recombined if the wallet needs a large amount.

Coinbase maturity also affects the workflow. A newly mined output is not immediately spendable. The wallet distinguishes pending and available balances according to maturity and the selected confirmation policy. Rotating coinbase addresses therefore creates a stream of pending outputs that become available later. This is the correct behavior for a public proof-of-work wallet, and it prevents the UI from encouraging instant churn of immature rewards.

19.10. Local Wallet Secret Protection and Datafile Privacy

On-chain privacy and local wallet privacy are related but distinct. If a user's seed is stolen, no amount of address rotation helps. Atho's alpha wallet includes several local controls for seed and datafile safety. `WalletSeed` uses a zeroizing wrapper and redacts debug output. `MnemonicPhrase` zeroizes on drop and redacts debug formatting by showing word count rather than phrase words. The Falcon code also uses zeroizing material around deterministic key generation and signing paths. These practices reduce accidental secret retention and accidental log exposure.

Wallet persistence uses a `.datafile` format. When the wallet is saved with a non-empty password, the state is encrypted with AES-256-GCM. The encryption key is derived from the password with PBKDF2-HMAC-SHA256, a random 16-byte salt, and `600,000` iterations. The datafile uses a random 12-byte

nonce and associated authenticated data built from the wallet datafile domain, password scheme, and version. Writes are atomic: data is written to a temporary file, synced, renamed, permissions are restricted, and the parent directory is synced when possible. On Unix, wallet files are set to owner-only permissions (`0600`).

The format also supports plaintext mode when the password is empty. That is a conscious implementation fact, not something to ignore. The desktop and runtime configuration default toward requiring wallet encryption, but the low-level format records plaintext mode honestly rather than pretending an empty password is encrypted. The white paper should therefore say both things: encrypted wallet files are implemented and normal UI flows require encryption, but empty-password plaintext is supported at the datafile layer and should be treated as an explicit operator choice with weaker protection.

Address labels, requested payment labels, recipient address book notes, and wallet names are local metadata. They are not part of consensus and do not appear as on-chain output data in the current transaction format. This is good for public-chain privacy. It also means local backups can contain sensitive context even when the chain does not. A backup JSON, text export, QR export, screenshot, or copied label can reveal relationships that the chain itself did not label. Local metadata privacy remains an operator responsibility.

19.11. Node, RPC, and API Boundaries Relevant to Privacy

The alpha node configuration defaults are privacy-relevant because wallets often leak through operational surfaces. RPC binds to loopback by default. API access is enabled for local use, public read-only behavior is separated from wallet and mining controls, admin access is disabled by default, wallet API access is disabled by default, and mining API access is disabled by default. RPC authentication supports cookie auth and HMAC-style persisted credentials. Command metadata classifies commands by permission, including wallet-secret, wallet-write, node-admin, test-only, and dangerous mainnet-blocked categories.

These controls do not hide chain data. They reduce the chance that a wallet-capable local node becomes an accidental remote wallet service. A public explorer can safely expose validated block and transaction data; it should not expose wallet secrets or mutation commands. A local wallet should not need to send seed material to a public API. Atho's architecture keeps wallet signing local and transaction submission separate from key custody. The node receives a final signed transaction, not the mnemonic phrase or wallet seed.

The wallet scan also depends on node readiness. Sending and mining are blocked until the local node is connected, running, synced, and the wallet has completed its initial scan. On mainnet and testnet, the desktop app also waits for at least one peer before sending or mining. This is partly reliability, but it prevents a user from acting on stale wallet state and accidentally creating odd transaction behavior during startup or sync. Stale-state transactions can become privacy leaks when users retry, double-build, or broadcast inconsistent attempts.

19.12. Current Limitations and Future Privacy Work

The alpha implementation should not be described as a shielded system. It is a transparent-chain wallet with good hygiene tools. The major limitations are clear:

- amounts are public
- input outpoints are public
- output lock digests are public
- txids and block placement are public
- common-input analysis remains effective
- change-output heuristics may still identify wallet change
- split outputs are siblings at creation

- self-sends do not erase graph history
- coinbase rotation can be undone by later consolidation
- no built-in network-origin anonymity is implemented
- no reusable silent-address or stealth-address receive scheme is implemented
- no shielded notes, nullifiers, commitments, or hidden-amount proofs are active consensus features

These limits define the development path beyond the alpha coin-control work. Future wallet improvements can deepen source grouping, add richer prepared-coin workflows, improve exact-match and no-change heuristics, add stronger acknowledgement policies for high-risk merges, and continue refining self-send/split workflows. Those features would not require a consensus change and would build directly on the existing `WalletSpendRequest` structure. Stronger receive privacy, such as silent-address-style reusable identifiers, would require more design work because the wallet must detect payments without forcing address reuse or leaking scanner state. Hidden amounts and shielded pools would require consensus-level cryptography and extensive audits, especially because Atho's post-quantum design goals rule out simply copying elliptic-curve privacy systems unchanged.

The alpha privacy message is therefore practical and modest: Atho already implements wallet hygiene features that many transparent chains need, and it does so in a way that is consistent with its HD wallet, Falcon signing, Base56 address, UTXO, mining, and local-node architecture. Users can avoid address reuse, rotate coinbase destinations, direct change away from receive addresses, split change into several future spend pieces, send to their own fresh addresses, select exact UTXOs, label and freeze coins locally, avoid mining/exchange/prepared merges, keep wallet signing local, encrypt wallet files, and rely on network-scoped address checks. They should not claim that those actions make the chain anonymous. They make the wallet less careless, and that is the correct foundation for stronger privacy layers later.

20. Advanced Coin Control in the Alpha Wallet

Advanced coin control is the wallet-facing feature set that turns Atho's transparent UTXO model into something users can actively manage. It does not change consensus, hide amounts, hide recipients, or create a mixing pool. Instead, it exposes the spendable-output facts that already exist in node and wallet state, adds local metadata that never goes on-chain, and routes manual selections into the existing strict `selected_utxos` transaction builder path.

The alpha implementation adds a compact coin-control panel inside the Send page. The panel lists wallet-owned UTXOs with amount, HD path, local label, output type, confirmation count, spend status, privacy status, selection state, and freeze controls. Rows are built from validated node UTXO state and locally derived wallet addresses. The table can show spendable, pending, reserved, frozen, and immature outputs, but only spendable, unfrozen, unreserved outputs can be selected for the current transaction.

```
Node UTXO set + wallet HD discovery
              |
              v
      Wallet-owned UTXO inventory
              |
              +-- local label / freeze / do-not-merge metadata
              |
              v
        Coin Control table in Send page
              |
      Automatic selection or manual selected set
              |
              v
        WalletSpendRequest.selected_utxos
```

```
        |
        v
local Falcon signing + transaction PoW
```

Local metadata is stored as a desktop-side coin-control JSON sidecar under the active Qt wallet root for the selected network. This is intentionally separate from consensus and separate from the encrypted wallet datafile format. Labels, frozen flags, and do-not-merge flags are local wallet hints. They are never serialized into transaction outputs, never sent to peers, and never become part of a block. If a user labels a coin `exchange`, `mining`, `split`, `private-prepared`, `personal`, or another local category, that label exists to help the wallet warn before accidental linkage. The chain still sees only ordinary transparent UTXOs.

The freeze control protects a UTXO from accidental spending. Frozen outputs remain visible in the inventory so the user can understand total wallet state, but they are excluded from automatic spend candidates and cannot be selected until unfrozen. This distinction matters: "frozen" is a user decision, "reserved" is a mempool or in-progress-send condition, and "immature" is a consensus condition such as coinbase maturity. The alpha GUI keeps those states separate instead of merging them into one vague unavailable balance.

Manual mode changes the spend path in a precise way. When manual coin control is disabled, the wallet uses its existing automatic UTXO selector, which sorts candidates, estimates transaction shape, respects dust and transaction policy, and prefers lower-fee, lower-input plans. When manual coin control is enabled, the selected outpoints become the candidate set, and the exact selected set is planned as a unit. The wallet does not silently add unrelated UTXOs to a manual transaction. If the selected set cannot cover recipients plus the selected fee target, or if a selected coin becomes frozen, reserved, immature, or spent before broadcast, the send is refused and the user must adjust the selection.

```
Automatic mode:
  spendable wallet UTXOs -> wallet selector -> chosen subset -> builder

Manual mode:
  user-selected UTXOs -> strict selected-set planner -> builder

Builder invariant:
  final transaction inputs == selected_utxos supplied by the send plan
```

The alpha wallet also adds smart buttons that help users reach better input sets without forcing them to solve coin selection by hand. `Exact` attempts a no-change plan for the current recipients. `Minimize change` uses the automatic selector to find a low-change candidate set and then pins that set into manual coin control. `Spend one` searches for a single UTXO that can cover the payment. `Prepared only` selects spendable coins labeled as split or prepared. `Avoid mining` removes coinbase and mining-labeled outputs from the current selection, or selects only non-mining outputs when no manual set exists yet. These are wallet conveniences, not consensus rules.

Change handling remains explicit. The Send page already supports wallet change, fresh unused wallet change addresses, custom change address validation, multiple recipients, tip-in-total mode, and split change output counts. Advanced coin control makes the input side visible beside those output controls. This lets the user see how a chosen input set interacts with change shape before signing. If the selected inputs make no-change possible, that is shown as the best linkability shape for that payment. If the selection creates large change, the user can choose a different set, use split change, or prepare coins for later.

The warning system is intentionally modest and local. It does not claim to make transactions private. It flags common transparent-chain mistakes before broadcast: combining different labels, spending from multiple wallet addresses, mixing mining rewards with other funds, linking exchange-known funds with prepared coins, spending weak change with fresher coins, or including a do-not-merge output. Warnings appear in the Send page workflow and are carried into the tip and final confirmation dialogs so the user sees them before transaction signing and

broadcast.

```
Selected inputs
   |
   +-- labels differ? -------------------- medium warning
   +-- multiple wallet addresses? -------- medium warning
   +-- mining + non-mining? -------------- high warning
   +-- exchange + prepared? -------------- high warning
   +-- weak change + other coins? -------- medium warning
   +-- do-not-merge selected? ------------ high warning
```

Mining rewards receive special handling because coinbase rotation can be undone by later spending behavior. Atho already supports rotating coinbase rewards to fresh wallet receive addresses in the desktop mining flow. Advanced coin control adds the next layer: coinbase outputs are identified in the UTXO inventory, automatically label as mining when no stronger local label exists, and trigger warnings when merged with other reward outputs or non-mining funds. This keeps the privacy story honest. Rotating reward addresses reduces receive-side clustering, but the spend path must still avoid recombining those rewards carelessly.

The implementation deliberately preserves full-node authority. The GUI can select, label, freeze, and preview, but it cannot bypass dust rules, optional-tip accounting rules, coinbase maturity, wrong-network address rejection, output-count limits, transaction-size limits, or final node validation. A selected UTXO is only useful if it remains a valid spendable output under the current chain and mempool view. A custom change address still must decode for the active network. A transaction still receives local Falcon signatures and transaction proof-of-work only after the final input/output shape is known.

This section should be read with the privacy boundary in Section 19. Advanced coin control improves basic user privacy by reducing avoidable linkability. It gives users visibility into UTXOs, lets them freeze sensitive coins, labels source categories locally, selects exact outputs, avoids accidental mining/exchange/prepared merges, and surfaces change risk before signing. It does not erase transaction graph history. It does not make self-splitting equivalent to CoinJoin. It does not prevent explorers from seeing public inputs, outputs, amounts, txids, block placement, or coinbase rewards. Its value is practical: it makes the safer wallet choice easier to see and easier to execute.

21. Address and Encoding Design

Atho uses user-facing Base56 address encoding derived from a 32-byte payment digest. The visible address carries a network-specific prefix and checksum. The internal consensus lock is not the visible string itself. The visible string is an interface encoding for the lock digest.

That separation matters:

- wallets and users interact with Base56 addresses
- consensus validates the canonical underlying digest
- network identity is visible in the address prefix
- decoding failures reject malformed user input before transaction construction

Table 11 summarizes the design tradeoffs of the current address format.

Table 11

Address Design Tradeoffs

Address Feature	Benefit	Constraint or Tradeoff
Base56 visible encoding	Avoids visually ambiguous characters and remains user-facing.	Slightly less conventional than Base58/Bech32 for outside tooling.
Visible network prefix	Makes network selection human-readable at the string level.	Wallet and exchange tooling must preserve prefix checks strictly.
32-byte payment digest	Matches canonical ownership binding in validation.	Visible address is an encoding of ownership, not ownership itself.
Fixed checksum suffix	Rejects malformed or mistyped input before transaction build.	Checksum is for input safety, not a substitute for consensus validation.
Separate internal lock vs. visible address	Keeps consensus bytes independent from UI encoding.	Developers must avoid treating display strings as canonical ledger state.

22. API and Developer Tooling

The project exposes multiple developer-facing layers:

- local launchers for `mainnet`, `testnet`, and `regnet`
- node APIs and RPC-like surfaces
- wallet tooling
- benchmarking and adversarial harness binaries
- desktop client integration

Developer tooling is valuable only when it routes into the same truth model as the node. Raw transaction submission must still hit canonical validation. Local mining helpers must still produce blocks that full validation accepts. Explorer endpoints must still reflect validated chainstate.

22.1. Practical API Boundaries

Public or semi-public APIs should:

- return validated chain and mempool information
- reject malformed raw transactions
- reject wrong-network payloads
- avoid exposing secret wallet material
- reflect the same confirmation and maturity policy enforced in code

23. Explorer and Indexer

Explorer and indexer surfaces are presentation layers over validated node data. Their role is to help users, exchanges, and operators inspect blocks, transactions, addresses, supply behavior, and network health. They do not define spendability, and they must not present invalid or legacy data as if it were canonical.

Because Atho already separates raw block archives from indexed metadata and UTXO state, explorer-style queries can be served from indexed products without reopening every raw historical block file for normal lookups.

24. Security Model

Atho’s security model combines several assumptions:

- proof-of-work ordering remains costly to rewrite without sufficient hashpower
- full nodes validate blocks and transactions independently
- Falcon-512 witness verification remains correctly implemented
- canonical ownership binding prevents loose script-based spending ambiguity
- network identities remain separated
- storage commits remain durable and coherent
- wallets protect secret material and do not leak seeds or signing keys

The security model should be described honestly. Atho is not “quantum-proof forever.” It is a payment chain whose transaction authorization model is built around a post-quantum signature scheme and whose consensus implementation is designed to fail closed on malformed or invalid state transitions.

Table 12 summarizes the main threat categories and the repository’s current defensive posture.

Table 12

Atho Threat Model

Threat Category	Defensive Posture	Remaining Dependency
Double-spend attempts	UTXO existence checks, duplicate-input rejection, and chain selection by validated proof-of-work.	Full nodes must remain widely deployed and honest enough to validate independently.
Unauthorized spend attempts	Canonical 32-byte lock binding plus Falcon-512 verification over rebuilt signing digests.	Wallets and nodes must continue to agree on exact signing-message bytes.
Wrong-network replay	Network-specific IDs, visible address prefixes, genesis anchors, and P2P magic values.	Operators must not intentionally disable network checks or mix state directories.
Malformed peer or API input	Strict decoders, size bounds, and typed validation errors.	Coverage still depends on continuing regression and adversarial testing.
Storage corruption or partial state drift	Hybrid block-file plus LMDB model with explicit chainstate indexing.	Crash-consistency and recovery behavior still need continuous validation.
Wallet secret exposure	Local signing, zeroizing memory for key material, password-based wallet encryption support, and UI/runtime defaults that require encryption.	The low-level datafile format still treats an empty passphrase as explicit plaintext mode, so operator flows should continue requiring encryption where appropriate.

25. Performance and Scalability

Performance in Atho comes from removing wasted work rather than weakening validation. Important hot paths include:

- canonical transaction decode
- UTXO lookups
- lock-digest ownership checks
- Falcon verification
- mempool conflict tracking
- block validation
- commitment-root reconstruction
- storage commits
- sync scheduling and relay

The project’s performance posture should continue to favor:

- batching safe work where possible
- caching exact validation products only when correctness is preserved
- avoiding redundant serializations
- limiting lock contention around mempool and storage access
- parallelizing independent witness verification without reordering final state transitions

26. Testing, Auditing, and Benchmarking

Atho benefits from layered verification:

- unit tests for protocol constants, encoding, hashing, signatures, and network identity
- integration tests for wallet, node, mempool, and storage paths
- adversarial tests for malformed inputs and replay attempts
- benchmark coverage for Falcon verification, decode paths, block validation, and storage commit behavior
- design reviews and audit reports for consensus correctness, Falcon handling, wallet behavior, storage behavior, and network behavior

This section documents verification areas rather than scoring the project. The repository’s tests and reports are useful only when they stay tied to the code paths they describe: transaction decoding, witness verification, UTXO transitions, storage recovery, chain selection, wallet construction, mining templates, API reporting, and network synchronization.

Table 13 lists the minimum testing areas that should remain in view for public network operation.

Table 13

Required Test Coverage Matrix

Test Area	What Must Hold	Why It Matters
Transaction decode	Malformed bytes reject deterministically	Prevents parser ambiguity and bypasses.
Ownership binding	Wrong key or wrong lock always fails	Protects spend authorization.
Monetary validation	Overpaying coinbase blocks reject	Preserves emission rules.
Network isolation	Wrong-network addresses and blocks reject	Prevents cross-network contamination.
Storage durability	Accepted state is reconstructable after restart	Preserves node trust in local chainstate.
Falcon verification	Malformed signatures and keys reject safely	Preserves authorization correctness.

Rust safety requirements also deserve explicit long-form tracking because the implementation is part of the security model, not just a vehicle for the protocol. Table 14 identifies the coding standards most relevant to consensus-critical review.

Table 14

Consensus-Critical Rust Review Standards

Rust Review Requirement	Why It Matters	Current Application Area
`#![forbid(unsafe_code)]` or equivalent in core crates	Shrinks the memory-unsafety surface for validation and state code.	`atho-core`, `atho-storage`, `atho-node`, and related binaries.
Typed enums for network and validation state	Prevents stringly typed rule branching.	Network IDs, versions, launch modes, and error surfaces.
Canonical serialization functions owned by protocol types	Prevents drift between wallet, node, and storage byte layouts.	Transactions, witnesses, blocks, addresses, and digests.
No panic-driven validation logic on hostile input	Keeps peer and API input from crashing the process.	P2P codec, address decode, raw transaction decode, and witness parsing.
Explicit activation scheduling for future rules	Makes dormant upgrades inspectable before activation.	Ruleset V2 placeholder and version checks.

27. Governance and Network Upgrade Mechanism

Atho separates software releases from network rule activation. Installing a new binary can change local software immediately, but a consensus change becomes active only when the compiled consensus schedule assigns that ruleset an explicit block height. A release tag, user-agent string, miner message, peer count, wall-clock date, or operator configuration flag does not by itself change block validity.

This distinction is central to safe network evolution. Every node validates the chain under its own installed rules. Developers can publish code and miners can produce blocks, but neither group has an administrative key that can order an independently validating node to accept a rule change. A network upgrade succeeds when operators voluntarily install compatible software before the deterministic activation point and the resulting chain satisfies the rules each upgraded node enforces.

27.1. Version Domains

Atho uses separate version numbers because the software package, peer protocol, consensus rules, block format, transaction format, and local database solve different compatibility problems. Table 15 identifies each domain and its current value.

Table 15

Atho Version Domains

Version Domain	Current Value	Purpose	Activation or Compatibility Rule
Software release	`0.1.0`	Identifies the built application and all workspace crates.	Read from the workspace package version and exposed in user-agent and RPC output; it does not activate consensus.
P2P protocol	`3`	Identifies peer message compatibility.	A handshake succeeds only when both peers' current and minimum supported versions overlap and all required services are present.
Consensus ruleset	`1`	Names the logical set of block and transaction validity rules.	Selected from the compiled fixed-height activation schedule using the candidate branch height.
Block version	`1`	Identifies the block-header version required by the active ruleset.	The version must equal the version selected for that exact block height.
Transaction version	`1`	Identifies the transaction version required by the active ruleset.	The version must equal the version selected for the validation or candidate-block height.
Storage schema	`6`	Identifies the local chain database layout.	A mismatched schema fails closed and requires a supported migration or a controlled rebuild/resync.
Network identity	Network-specific	Separates mainnet, testnet, regnet, and prunetest histories.	Genesis hash, network ID, P2P magic, ports, and address prefixes must agree; changing identity creates another network rather than silently upgrading the existing chain.

The software version is therefore not a substitute for any protocol version. Two releases may share consensus rules while differing in user interface or performance, and a consensus release may need coordinated changes across the node, miner, wallet, explorer, and integration tooling even when those components display one common package version.

27.2. Implemented Fixed-Height Activation

The current implementation uses a deterministic fixed-height schedule. The schedule is compiled into consensus code as an ordered array of entries. Each entry contains:

- a unique, stable ruleset name
- a unique ruleset version
- the exact required block version
- the exact required transaction version
- an activation height, or `None` while the entry is dormant

V1 is active from height `0`. The V2 placeholder has ruleset version `2`, block version `2`, and transaction version `2`, but its activation height is `None`. A dormant entry is visible for review but never becomes active. The placeholder does not yet define a complete second generation of consensus behavior and must not be described as a deployed upgrade.

Schedule validation fails closed. A canonical schedule must begin at height `0`, use non-empty names, keep names and all version numbers unique, order active heights strictly upward, and never place an active entry after a dormant entry. Rule resolution considers only entries whose activation height is present and less than or equal to the candidate height, then selects the latest such entry. This makes the result independent of wall-clock time and peer opinion.

The candidate branch height is authoritative. During a reorganization, blocks are checked under the rules assigned to each height on the candidate branch; the node does not retain a mutable global switch that can drift away from chain context. Transactions and blocks carrying future versions are rejected before activation. At activation, the old versions cease to be the canonical versions for that height.

Atho does not currently implement Bitcoin-style version-bit voting or a miner-signaled lock-in state machine. Miner signaling has no consensus effect. A future signaling mechanism would itself be a consensus feature requiring a separate specification, implementation, activation, and audit. It must not be inferred from header versions or mining participation.

27.3. Upgrade Lifecycle and Release Gates

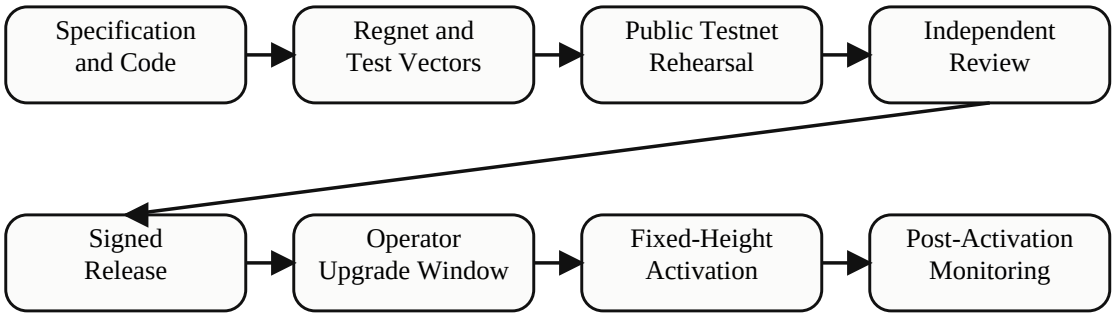


Figure 14. Atho Fixed-Height Network Upgrade Lifecycle

A consensus deployment should move through the following lifecycle. These are release and coordination requirements around the implemented fixed-height mechanism; the social review stages are not automated by chain consensus.

1. **Specification:** publish the exact old and new rules, affected serialization, activation semantics, security assumptions, and expected behavior of upgraded and unupgraded nodes.
2. **Classification:** identify whether the change is release-only, relay policy, P2P protocol, storage schema, fixed-height consensus, or a new-network reset. Ambiguous changes do not proceed.
3. **Dormant implementation:** add the complete rules behind a named schedule entry while leaving mainnet activation unset. Keep consensus constants centralized and eliminate alternate code paths that could disagree.
4. **Deterministic vectors:** publish valid and invalid blocks and transactions immediately before, at, and after the proposed boundary. Include reorgs that cross the boundary in both directions.
5. **Regnet and prunetest:** test activation, restart, pruning, snapshot import, chainstate reconstruction, mempool handling, mining templates, and wallet transaction creation in controlled networks.
6. **Public testnet rehearsal:** activate the same rule behavior on testnet at a testnet-specific height, observe mixed-version peers, and repeat recovery and rollback drills.
7. **Independent review:** obtain consensus, cryptography, storage, networking, and wallet review appropriate to the affected surface. Resolve or explicitly disclose findings.
8. **Release freeze:** freeze rule bytes, versions, test vectors, documentation, and the proposed mainnet height. A change after freeze creates a new candidate and repeats the affected gates.
9. **Signed release:** publish signed source tags, binaries, checksums, the release-key fingerprint, schema instructions, minimum peer version, and the exact activation manifest.
10. **Upgrade window:** allow enough block and calendar time for nodes, miners, wallets, exchanges, explorers, and pools to install and verify the release. Monitor readiness externally; no observed percentage overrides consensus.
11. **Activation:** upgraded miners build the exact active block and transaction versions, while upgraded nodes enforce the new rules at the fixed height.

12 **Post-activation monitoring:** compare independently operated nodes for tip, chainwork, ruleset, peer compatibility, mempool health, reorgs, and invalid-block reports. Preserve release evidence and incident logs.

27.4. Upgrade Classes

Not every update requires a network fork. Table 16 defines the operational classification that should be applied before implementation.

Table 16

Network Upgrade Classes

Upgrade Class	Examples	Network Effect	Required Treatment
Release-only	GUI changes, diagnostics, performance improvements that preserve deterministic results	No change to valid blocks or transactions	Normal signed release and regression testing.
Local policy	Relay limits, peer scoring, wallet confirmation preferences	Nodes may relay or display data differently but must agree on block validity	Document as non-consensus and test that policy cannot alter historical validation.
P2P protocol	New messages, framing, required services, minimum peer version	Incompatible peers may disconnect while still recognizing the same chain	Bump protocol compatibility fields when required and preserve explicit fallback behavior where supported.
Storage schema	New indexes, persisted fields, chainstate representation	Changes local files, not ledger validity	Bump schema version and provide a tested migration or documented rebuild/resync path.
Consensus tightening	A new restriction on blocks previously accepted by older nodes	Unupgraded nodes may accept blocks upgraded nodes reject	Schedule explicitly and treat as fork-sensitive even when sometimes described as soft-fork-style.
Consensus expansion or format change	New block/transaction version or newly valid behavior	Older nodes reject blocks accepted by upgraded nodes	Mandatory fixed-height upgrade with full ecosystem coordination.
Network identity reset	New genesis, chain ID, P2P magic, or address identity	Creates an independent history	Use only for a deliberate new network or pre-launch reset; never present it as an in-place mainnet upgrade.

The current V2 placeholder would require exact block version `2` and transaction version `2`. Because V1 nodes require version `1`, a deployment using that transition is mandatory-upgrade, hard-fork-style behavior: old nodes will reject the V2 chain at activation. Calling it a soft fork would be inaccurate. A future restriction that truly remains valid to old nodes could have soft-fork-style compatibility, but it would still require an explicit Atho specification and schedule rather than relying on terminology alone.

27.5. Node Behavior Around an Activation

Before activation: upgraded nodes continue enforcing the prior ruleset. Future-version blocks and transactions remain invalid. If the P2P protocol range and required services overlap, old and new software can remain connected. The scheduled entry is visible through node diagnostics, allowing operators to confirm the planned height before it is reached.

For the activation block: miners derive the next height from the validated tip, select the active rules for that height, and construct the required block header and coinbase transaction versions. Candidate selection revalidates mempool entries for the candidate height and skips entries that are stale or invalid under those rules. Full nodes independently apply the same height-derived checks to the header, transactions, witnesses, UTXO transitions, proof of work, and monetary accounting.

After activation: every later block remains subject to the latest scheduled rules whose height has been reached. A reorganization that crosses the boundary evaluates every candidate-branch block under the schedule entry for that block's height while reversing the prior branch through deterministic undo state. There is no grace period, peer-majority exception, checkpoint override, or runtime flag that makes a wrong-version block valid.

An unupgraded V1 node will reject a V2 activation block because the block and transaction versions do not match its compiled schedule. Once a V2 peer advertises V2 as the ruleset appropriate for its best height, the V1 peer can also reject the handshake as a ruleset mismatch. If old miners continue extending V1, they can create a minority fork that upgraded nodes reject. This is why activation readiness and hash-power distribution must be observed before the fixed height even though those observations do not vote the rule into effect.

Wallets, exchanges, merchant systems, explorers, pools, and offline signers must be upgraded when their generated or interpreted data changes. The current V1 wallet constructs V1 transactions; a V2 deployment must update transaction construction and signing before activation. It is not sufficient to upgrade only block-producing nodes.

27.6. P2P Compatibility and Network Isolation

The version handshake carries the current protocol version, the minimum protocol version the sender can support, required service bits, network identity, genesis hash, best height, active ruleset version for that height, tip hash, and chainwork. A peer is rejected when:

- the selected network or genesis hash differs
- protocol compatibility ranges do not overlap
- mandatory full-block or full-witness services are missing
- the advertised ruleset does not match the receiving node's compiled expectation for the advertised height

The active P2P protocol version and minimum supported version are both `3` in the current line. A future wire-compatible release may retain them. A wire-incompatible release must define the new version, minimum compatibility boundary, message behavior, and fallback strategy. Raising the minimum version isolates old peers but does not by itself activate new consensus rules.

Mainnet, testnet, regnet, and prunetest remain isolated by more than software version. They have distinct network IDs, genesis anchors, P2P magic values, ports, storage roots, and visible address prefixes. Data from one network must not become an upgrade path into another.

27.7. Storage and Local-State Upgrades

Storage schema versioning protects local chainstate from being interpreted under the wrong layout. The database records the expected network and genesis identity together with schema metadata. A schema mismatch fails closed rather than guessing how to decode state. The runtime recovery path can quarantine incompatible or corrupt storage before rebuilding, but operators should still back up wallet data and configuration and follow release-specific migration or resynchronization instructions.

A storage change is not automatically a consensus change. Indexes and cache layouts may change while the accepted chain remains identical. Conversely, changing consensus constants without changing the schema does not make old chain data valid under the new rules. Release notes must independently identify:

- whether the database can be opened in place
- whether an explicit migration exists
- whether a full resync is required
- whether snapshots remain compatible
- whether downgrading the binary can still read the upgraded database

Wallet backups are separate from chainstate backups. A node can resynchronize public chain data, but it cannot reconstruct lost private keys or mnemonic passphrases from the network.

27.8. Release Manifest and Operator Verification

Every consensus release should publish one unambiguous activation manifest containing:

- network name and genesis hash
- source commit and signed release tag
- software, P2P protocol, minimum protocol, ruleset, block, transaction, and storage versions
- testnet and mainnet activation heights
- hash or exact location of the reviewed specification and test vectors
- required wallet, miner, pool, explorer, and exchange versions
- migration, resync, mempool, backup, and downgrade instructions
- release checksums, detached signature, and independently published signing-key fingerprint

The repository provides deterministic checksum-manifest tooling and signed-tag instructions. The production release key and independently published fingerprint are not yet finalized, so the process is documented but not yet a completed mainnet trust root. The activation manifest is an operational artifact; current consensus does not download or trust a remote manifest automatically.

Operators can inspect local state through `getversion`, `getrulesetinfo`, `getconsensusstatus`, network diagnostics, and peer diagnostics. `getrulesetinfo` reports the active protocol, ruleset, block, and transaction versions together with the complete compiled schedule. Operators should compare this output across independently built nodes before and after activation rather than trusting one hosted endpoint.

When a transaction-version change is scheduled, operators should expect old-version pending transactions to stop qualifying for activation-height blocks. Wallet and exchange systems should stop creating the old format before the boundary, rebroadcast or rebuild transactions only under reviewed rules, and never mutate a signed transaction while assuming its original transaction identity or proof-of-work remains valid.

27.9. Rollback and Emergency Semantics

Before activation, a proposed height can be canceled or replaced only by publishing another reviewed release early enough for operators to adopt the revised compiled schedule. Competing releases that assign different rules to the same height are a chain-split risk. A last-minute schedule change is therefore an emergency coordination event, not an ordinary configuration update.

After activation, operators must not downgrade to software that lacks the active rules. A binary rollback is safe only when the rollback build still understands and enforces every rule already active on the accepted chain and can read the current storage schema. Restoring an old executable or database backup does not reverse consensus history. If an activated rule must later change, the network needs another explicit upgrade at a later height or, in an extreme failure, a separately specified recovery fork.

Atho has no central pause key, administrator override, or developer command that can rewrite an independently validated chain. Emergency releases follow the same signed distribution and local-validation model. Checkpoints and trusted snapshots may accelerate or anchor synchronization according to configured policy, but they do not authorize an otherwise invalid post-activation block.

Genesis or chain-identity changes are not rollback tools. Before a real-value launch, a project may deliberately reset an experimental network and publish new genesis anchors. After launch, changing genesis establishes a different network and leaves the original history governed by the software its participants continue to run.

27.10. Governance Responsibilities and Current Status

Atho currently has no on-chain governance vote and no miner-signaled activation state machine. Governance is the transparent process by which specifications are proposed, code is reviewed, releases are signed, and independent operators decide which software to run. Responsibilities are distributed:

- maintainers define the candidate rules precisely, keep code and documentation aligned, and publish signed artifacts
- reviewers audit consensus, cryptography, storage, networking, wallet, and integration effects
- miners and pools upgrade template construction and avoid producing obsolete-version blocks
- node operators verify release signatures, back up local state, test upgrades, and monitor their own validated tip
- wallets and hardware signers produce the active transaction format without exposing keys
- exchanges and merchants rehearse deposits, withdrawals, confirmations, reorgs, maintenance windows, and recovery
- explorers and monitoring services report ruleset and chain data without becoming sources of consensus truth

The deployed mainnet schedule currently contains only one active entry: V1 at height `0`. V2 remains dormant with no activation height and no completed V2 semantic rules. The mechanism for deterministic fixed-height activation is implemented and tested; the first future activation deployment, production signing identity, external review record, adoption window, and incident ownership must be completed when an actual V2 proposal exists.

The upgrade philosophy should remain conservative:

- keep consensus rules centralized and height-derived

- make every consensus change explicit in versions, tests, and release documentation
- avoid ambiguous compatibility layers and runtime consensus toggles
- treat genesis and network identity changes as new histories
- keep wallet, API, explorer, storage, P2P, and mining behavior aligned with node validation
- preserve operator choice and independent validation throughout deployment

28. Development and Deployment Status

The current repository supports local development, regnet, and extended testnet operation. Mainnet-style production deployment remains a no-go. Independent consensus and cryptographic review, reproducible signed releases, dependency and provenance gates, sustained hostile-network testing, cross-platform recovery exercises, and production incident ownership are not complete. The production deployment guide and testing document maintain the detailed, current release blockers.

The following code and documentation surfaces remain part of ongoing maintenance:

- maintain public network configuration for mainnet, testnet, regnet, and prunetest
- keep wallet, node, miner, explorer, and API behavior aligned with consensus validation
- continue fuzz, differential, restart, reorg, storage, and network-synchronization testing
- document operator configuration, mining reward-address rules, RPC authentication, and wallet backup behavior
- maintain exchange, merchant, explorer, and application integration surfaces through typed RPC/API responses
- keep public documents synchronized with consensus constants, monetary policy, transaction policy, and block-sizing rules
- keep wallet privacy documentation aligned with implemented address rotation, change handling, transaction construction, and mining reward behavior

29. Conclusion

Atho is designed as a proof-of-work payment network whose strongest traits are not maximal complexity, but explicitness, deterministic validation, post-quantum-aware transaction authorization, and operator-reviewable rules. The current implementation uses SHA3-384 mining, Falcon-512 witnesses, canonical ownership binding, Base56 visible addresses, network-specific genesis anchors, 100-second blocks, 1-confirmation normal transaction consensus spendability, 100-block coinbase maturity, a zero-emission genesis anchor, a four-era `8/4/2/1 ATHO` bootstrap schedule, a permanent `0.50 ATHO` tail reward, 8 decimal places, no premine, no maximum supply limit, and an adaptive block limit that launches at 1,000,000 bytes while remaining bounded by the 10,000,000-vbyte and 10,000,000-raw-byte hard ceilings.

The codebase presents a payment-focused Layer 1 design: UTXO accounting, locally verifiable proof-of-work, integer-only monetary rules, Falcon-512 authorization, explicit network identities, wallet-side transaction construction, node-side consensus validation, wallet-level privacy hygiene, and indexed explorer/API reporting. The paper’s role is to describe those facts and keep the written model aligned with the repository.

References

1 Atho contributors. *Atho reference implementation and documentation, current repository revision.*

2 S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System.* 2008. https://bitcoin.org/bitcoin.pdf

3 National Institute of Standards and Technology. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions.* FIPS 202, 2015. https://csrc.nist.gov/pubs/fips/202/final

4 National Institute of Standards and Technology. *Post-Quantum Cryptography Standardization Process.* FIPS 206 / FN-DSA status. https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization

5 Falcon Team. *Falcon: Fast-Fourier Lattice-Based Compact Signatures over NTRU.* https://falcon-sign.info/

6 Rust Project Developers. *The Rust Programming Language.* https://doc.rust-lang.org/book/

7 LMDB project. *Lightning Memory-Mapped Database documentation.* https://www.openldap.org/software/repo.html

8 Thomas Pornin. *rust-fn-dsa.* Upstream implementation metadata for Atho's vendored `fn-dsa` code. https://github.com/pornin/rust-fn-dsa

Appendix A. Glossary

- **atom:** the smallest indivisible accounting unit in Atho; 1 ATHO equals 100,000,000 atoms.
- **Base56:** the user-facing visible address alphabet and encoding used by Atho addresses.
- **change address:** a wallet-controlled address used to receive the unspent remainder of a transaction.
- **coinbase maturity:** the number of confirmations required before a mined subsidy output may be spent.
- **Falcon-512:** the lattice-based post-quantum signature scheme used for Atho transaction authorization.
- **founder-hash metadata:** fixed `SHA3-384` and `SHA3-512` fields included in canonical Atho block headers.
- **HD wallet:** a deterministic wallet that can reproduce receive and change address sequences from seed material.
- **LMDB:** the key-value store used for indexed chainstate and metadata persistence.
- **linkability:** the ability of an observer to infer that addresses, outputs, or transactions share control or context.
- **bootstrap-plus-tail emission:** a monetary design where reward-bearing blocks follow explicit bootstrap reward eras and then continue under a fixed permanent tail reward rather than ending at a zero-subsidy hard cap.
- **UTXO:** unspent transaction output; the accounting object that can later be spent exactly once.

Appendix B. Protocol Constants

Table 17

Protocol Constants by Network

Constant	Mainnet	Testnet	Regnet	Prunetest
Consensus identity	Distinct mainnet ID	Distinct testnet ID	Distinct regnet ID	Distinct prunetest ID
Target block time	100 seconds	100 seconds	100 seconds	100 seconds
Max block virtual size	10,000,000 vbytes	10,000,000 vbytes	10,000,000 vbytes	10,000,000 vbytes
Max block weight	40,000,000 weight units	40,000,000 weight units	40,000,000 weight units	40,000,000 weight units
Max serialized block size	10,000,000 bytes	10,000,000 bytes	10,000,000 bytes	10,000,000 bytes
Sizing model	SegWit-style vbytes	SegWit-style vbytes	SegWit-style vbytes	SegWit-style vbytes
Normal transaction consensus spendability	1 confirmation	1 confirmation	1 confirmation	1 confirmation
Default wallet confirmation filter	3 confirmations	3 confirmations	3 confirmations	3 confirmations
Coinbase maturity	100 blocks	100 blocks	100 blocks	100 blocks
Proof-of-work hash	SHA3-384	SHA3-384	SHA3-384	SHA3-384
Signature scheme	Falcon-512	Falcon-512	Falcon-512	Falcon-512
Address format	Base56 with mainnet prefix	Base56 with testnet prefix	Base56 with regnet prefix	Base56 with prunetest prefix
Founder hashes	Fixed canonical header values	Fixed canonical header values	Fixed canonical header values	Fixed canonical header values

Appendix C. Code Reference Map

Table 18

Code Reference Map

Module or File	Role in the Current Repository
`crates/atho-core/src/constants.rs`	Consensus constants for timing, subsidy, confirmations, and units.
`crates/atho-core/src/network.rs`	Network IDs, visible prefixes, ports, P2P magic values, and network parameter boundaries.
`crates/atho-core/src/block.rs`	Canonical block header encoding, founder-hash fields, and block serialization.
`crates/atho-core/src/transaction.rs`	Canonical transaction and witness object model.
`crates/atho-core/src/address.rs`	Base56 address derivation, checksum logic, and decode rules.
`crates/atho-core/src/version.rs`	Workspace-driven software version and user-agent construction.
`crates/atho-core/src/consensus/rules.rs`	Active protocol/ruleset versions and future placeholder scheduling.
`crates/atho-core/src/consensus/subsidy.rs`	Bootstrap-plus-tail reward policy, annual issuance helpers, tail activation math, and cumulative supply reporting.
`crates/atho-core/src/consensus/pow.rs`	Difficulty targeting and proof-of-work validation rules.
`crates/atho-core/src/consensus/signatures.rs`	Canonical signing-message construction and witness verification context.
`crates/atho-crypto/src/falcon.rs`	Falcon-512 key and signature wrappers plus verification behavior.
`crates/atho-storage/src/validation.rs`	Contextual transaction and block validation against chainstate.
`crates/atho-storage/src/block_files.rs`	Flat block archive layout, file rotation, and offset bookkeeping.
`crates/atho-wallet/src/wallet.rs`	Wallet state, address derivation, explicit selected-UTXO spend requests, change splitting, and transaction construction.
`crates/atho-wallet/src/hd.rs`	HD wallet receive/change path model and seed wrapper.

Module or File	Role in the Current Repository
`crates/atho-wallet/src/keypool.rs`	Receive/change keypool reservation and refill behavior.
`crates/atho-wallet/src/wallet/datafile.rs`	Password-backed wallet datafile encryption, plaintext mode, atomic writes, and owner-only file permissions.
`crates/atho-qt/src/app/pages/send.rs`	Desktop send controls for recipients, tip handling, change mode, and change-output count.
`crates/atho-qt/src/app/pages/receive.rs`	Receive-address generation, QR output, request history, and address pool usage display.
`crates/atho-node/src/mining.rs`	Mining template construction and network-checked reward address decoding.
`crates/atho-node/src/api.rs`	Node-facing API surface and reporting.
`crates/atho-node/src/service.rs`	RPC commands for software, consensus, ruleset, network, and peer-version diagnostics.
`crates/atho-p2p/src/config.rs`	Network bootstrap configuration, current/minimum protocol versions, and peer limits.
`crates/atho-p2p/src/protocol.rs`	Version-handshake validation for protocol range, services, network identity, genesis, and active ruleset.
`crates/atho-storage/src/db.rs`	Storage schema checks, network/genesis metadata, and fail-closed database opening.
`crates/atho-rpc`	Typed local RPC transport and request/response model.
`crates/atho-installer`	Release installation and distribution tooling.
`scripts/release_checksums.py`	Deterministic signed-release checksum manifest creation and verification.

Appendix D. Transaction Validation Pseudocode

```
function validate_transaction(raw_bytes, network, utxo_view):
    tx = decode_canonical_transaction(raw_bytes)
    reject_if_trailing_bytes(raw_bytes, tx)
    reject_if_wrong_network(tx, network)
    reject_if_duplicate_inputs(tx.inputs)
    reject_if_invalid_output_amounts(tx.outputs)
    reject_if_invalid_tx_pow(tx)

    resolved_inputs = batch_lookup_utxos(tx.inputs, utxo_view)
    reject_if_missing_inputs(resolved_inputs)
    reject_if_immature_coinbase_spend(resolved_inputs)

    for input in tx.inputs:
        utxo = resolved_inputs[input]
        witness = parse_witness(input.witness)
        reject_if_malformed_witness(witness)
        reject_if_lock_not_canonical(utxo.lock_digest)
        reject_if_public_key_does_not_match_lock(witness.public_key, utxo.lock_digest)
        digest = rebuild_signing_digest(tx, input.index, utxo, network)
        reject_if_falcon_verification_fails(witness.public_key, witness.signature, digest)

    reject_if_created_value_exceeds_available_value(tx, resolved_inputs)
    return validated_transaction
```

Appendix E. Block Validation Pseudocode

```
function validate_block(raw_block_bytes, network, chainstate):
    block = decode_canonical_block(raw_block_bytes)
    reject_if_wrong_network(block.header, network)
    reject_if_bad_header_structure(block.header)
    reject_if_invalid_pow(block.header)
    reject_if_bad_timestamp_or_target(block.header, chainstate)
    reject_if_bad_coinbase_position(block.transactions)
    reject_if_duplicate_txids(block.transactions)
    reject_if_duplicate_spends(block.transactions)

    utxo_batch = batch_lookup_all_inputs(block.transactions, chainstate)
    optional_tips = 0
    for tx in block.transactions excluding coinbase:
        validated_tx = validate_transaction(tx.raw_bytes, network, utxo_batch)
        optional_tips += validated_tx.optional_tip

    reject_if_coinbase_overpays(block.coinbase, block.height, optional_tips)
    reject_if_bad_commitment_roots(block)
    atomically_apply_block(block, utxo_batch, chainstate)
    return accepted_block
```

Appendix F. Mempool Admission Pseudocode

```
function admit_to_mempool(raw_tx_bytes, network, chainstate, mempool):
    validated_tx = validate_transaction(raw_tx_bytes, network, chainstate.utxo_view)
    reject_if_mempool_conflict(validated_tx.inputs, mempool)
    reject_if_policy_invalid(validated_tx)
    mempool.insert(validated_tx)
    return admitted
```

Appendix G. Flowchart Source Text

The figures in this edition are rendered locally as a mix of report-style box diagrams and monospaced text flowcharts. Their intended meanings are summarized below.

Figures 1-5. Figure 1 follows wallet creation through mempool admission, mining, durable storage, and downstream sync. Figure 2 separates node responsibility boundaries. Figure 3 follows canonical signing through node-side Falcon verification. Figure 4 covers the transaction lifecycle through settlement. Figure 5 charts adaptive block-limit growth under sustained demand.

Figures 6-10. Figure 6 covers bootstrap issuance, tail activation, and long-horizon supply growth. Figure 7 separates the required fee, optional tip, and TX-PoW admission cost. Figure 8 follows mempool admission. Figure 9 shows ordered block validation. Figure 10 identifies parallel validation work and the ordered final commit.

Figures 11-13. Figure 11 covers node synchronization and block propagation. Figure 12 shows the hybrid block-file and LMDB commit model. Figure 13 covers wallet-to-node interaction from seed material to validated status.

Figures 14-17. Figure 14 defines the alpha wallet privacy boundary between local controls and public-chain facts. Figure 15 separates HD receive and change branches. Figure 16 follows the send builder through split change, grouped Falcon signing, and transaction proof of work. Figure 17 shows coinbase reward-address rotation after accepted mined blocks.

Two inline flowcharts supplement them. **Adaptive Block-Limit Epoch Flow** summarizes the `864`-block expand, shrink, or hold rule under fixed hard caps. **Canonical Tip Extension and Competing Branch Flow** applies the active limit to direct tip extensions and buffers competing branches for work-and-ancestry evaluation.